Advanced Machine Learning, ETH Zurich

Attention and transformers

From one attention map to a trained transformer

Carlos Cotrini

Autumn semester 2026

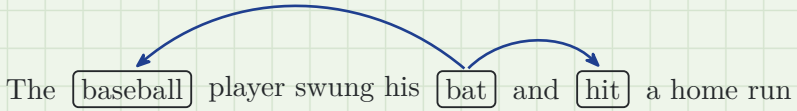
Entry 1

Attention

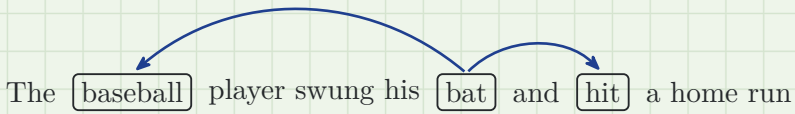
Words can find their meaning in a sentence by **attending to** other words!

The baseball player swung his bat and hit a home run

Words can find their meaning in a sentence by **attending to** other words!

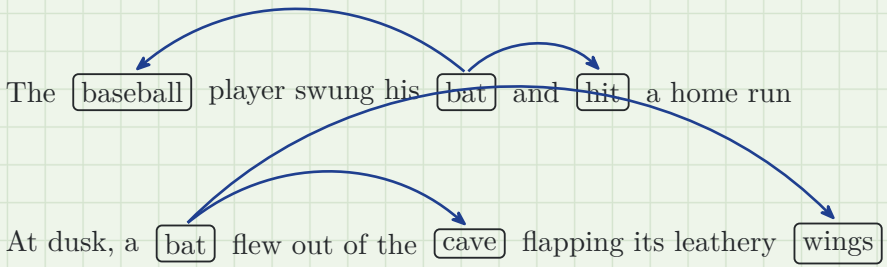


Words can find their meaning in a sentence by **attending to** other words!



At dusk, a bat flew out of the cave flapping its leathery wings

Words can find their meaning in a sentence by **attending to** other words!

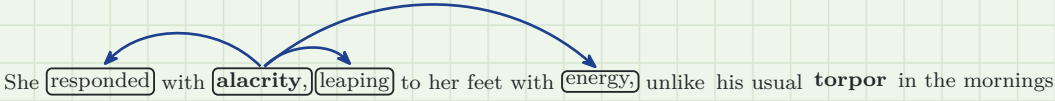


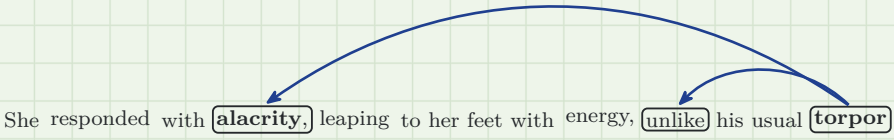
- She responded with **alacrity**, leaping to her feet with energy, unlike his usual **torpor** in the mornings.
- What do the words “alacrity” and “torpor” mean?





- She responded with **alacrity**, leaping to her feet with energy, unlike his usual **torpor** in the mornings.
- What do the words “alacrity” and “torpor” mean?
- Alacrity: brisk and cheerful readiness.
- Torpor: a state of physical or mental inactivity; lethargy.


She responded with alacrity, leaping to her feet with energy, unlike his usual torpor in the mornings.


She responded with alacrity, leaping to her feet with energy, unlike his usual torpor in the mornings.

- We **define** a **value** representation for each word.

- We **define** a **value** representation for each word.
- We let other words **enrich** their representations by **attending** to other words.

- Wuggish, a toy language: every sentence describes at most three things.

- Wuggish, a toy language: every sentence describes at most three things.



streept kat



bootet kat

- Wuggish, a toy language: every sentence describes at most three things.



streept kat



vennomus snaak wudden chaar



bootet kat



streept snaak streept kat wudden chaar

- Wuggish, a toy language: every sentence describes at most three things.



streept kat



bootet kat

Eight words

kat	cat
snaak	snake
chaar	chair
streept	striped
bootet	booted
vennomus	venomous
wudden	wooden
brooken	broken



vennomus snaak wudden chaar



streept snaak streept kat wudden chaar

- Learn the eight words from pictures, then read bags of words in any order.

wudden *snaak* *vennomus* *chaar*



https://eth-cscs-hs26.github.io/CSCS-PAGE/viz/part1/wuggish/?dev=0

- There are 8 words.
- There are 124 possible sentences.
- How can we characterize each sentence?

- There are 8 words.
- There are 124 possible sentences.
- How can we characterize each sentence?



vennomus streept snaak wudden chaar



kat brooken chaar

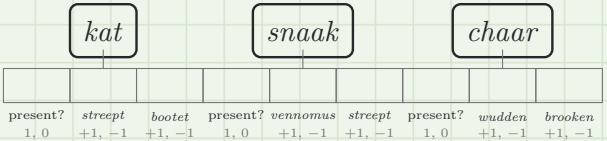
- There are 8 words.
- There are 124 possible sentences.
- How can we characterize each sentence?



vennomus streept snaak wudden chaar



kat brooken chaar



- There are 8 words.
- There are 124 possible sentences.
- How can we characterize each sentence?



vennomus streept snaak wudden chaar



kat brooken chaar

	kat			*snaak*			*chaar*		
	present?	*streept*	*bootet*	present?	*vennomus*	*streept*	present?	*wudden*	*brooken*
	1, 0	+1, −1	+1, −1	1, 0	+1, −1	+1, −1	1, 0	+1, −1	+1, −1
A	0	0	0	1	+1	+1	1	+1	−1

- There are 8 words.
- There are 124 possible sentences.
- How can we characterize each sentence?



vennomus streept snaak wudden chaar



kat brooken chaar

	kat			*snaak*			*chaar*		
	present?	*streept*	*bootet*	present?	*vennomus*	*streept*	present?	*wudden*	*brooken*
	1, 0	+1, −1	+1, −1	1, 0	+1, −1	+1, −1	1, 0	+1, −1	+1, −1
A	0	0	0	1	+1	+1	1	+1	−1
B	1	−1	−1	0	0	0	1	−1	+1


Query

Query





Keys

GreenChoice Supermarket



StoryStack Books



Threadline Clothing Co



Candy Cave



Spirit Cellar Liquors



Query





Keys

GreenChoice Supermarket



StoryStack Books



Threadline Clothing Co



Candy Cave



Spirit Cellar Liquors



Values











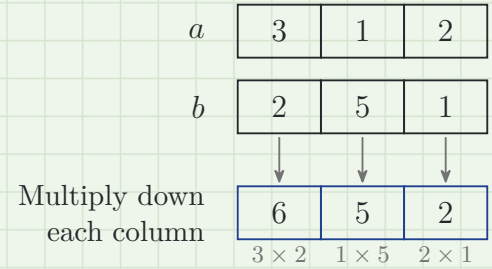


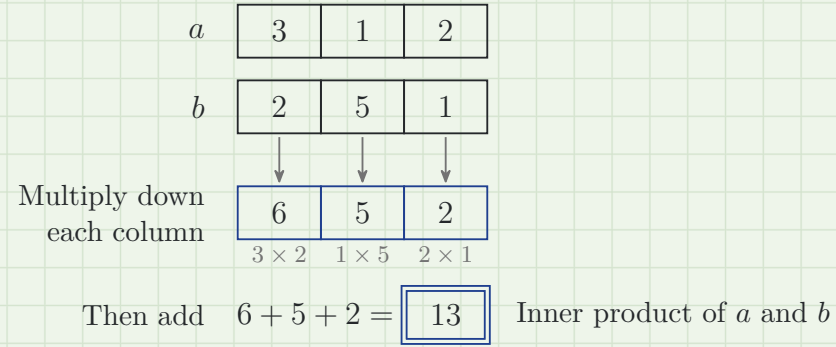
Entry 2

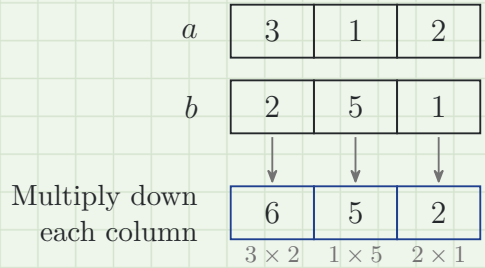
The inner product

a | 3 | 1 | 2 |

a	3	1	2
b	2	5	1


a
3
1
2
b
2
5
1
Multiply down each column
6
5
2
3 × 2
1 × 5
2 × 1

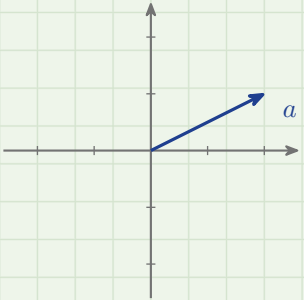

a
3
1
2
b
2
5
1
Multiply down each column
6
5
2
3 × 2
1 × 5
2 × 1
Then add
6 + 5 + 2 =
13
Inner product of a and b

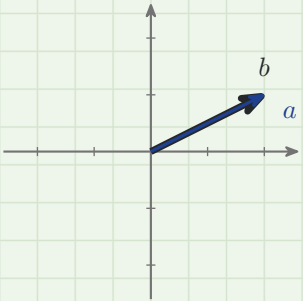


Then add $6 + 5 + 2 = \boxed{13}$ Inner product of a and b

Inner product of a and $b = a_1b_1 + a_2b_2 + a_3b_3$

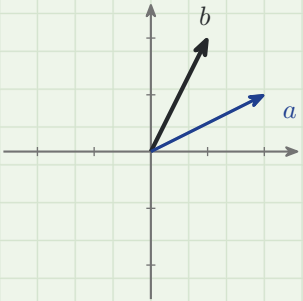
$a = (2, 1)$





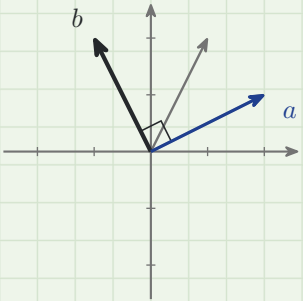
$a = (2, 1)$

$b = (2, 1)$ $\quad 2 \times 2 + 1 \times 1 = 5$ Same direction



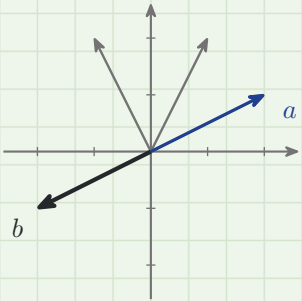
$a = (2, 1)$

$b = (2, 1)$	$2 \times 2 + 1 \times 1 = 5$	Same direction
$b = (1, 2)$	$2 \times 1 + 1 \times 2 = 4$	Close directions



$a = (2, 1)$

$b = (2, 1)$	$2 \times 2 + 1 \times 1 = 5$	Same direction
$b = (1, 2)$	$2 \times 1 + 1 \times 2 = 4$	Close directions
$b = (-1, 2)$	$2 \times (-1) + 1 \times 2 = 0$	At right angles



$a = (2, 1)$

$b = (2, 1)$	$2 \times 2 + 1 \times 1 = 5$	Same direction
$b = (1, 2)$	$2 \times 1 + 1 \times 2 = 4$	Close directions
$b = (-1, 2)$	$2 \times (-1) + 1 \times 2 = 0$	At right angles
$b = (-2, -1)$	$2 \times (-2) + 1 \times (-1) = -5$	Opposite directions


kat
snaak
chaar
Query of kat
1
0
0

	kat	*snaak*	*chaar*	Inner product
Query of *kat*	1	0	0	
Key of *kat*	1	0	0	**1**

	kat	*snaak*	*chaar*	Inner product
Query of *kat*	1	0	0	
Key of *kat*	1	0	0	**1**
Key of *bootet*	1	0	0	**1**

	kat	*snaak*	*chaar*	Inner product
Query of *kat*	1	0	0	
Key of *kat*	1	0	0	**1**
Key of *bootet*	1	0	0	**1**
Key of *wudden*	0	0	1	**0**

	kat	*snaak*	*chaar*	Inner product
Query of *kat*	1	0	0	
Key of *kat*	1	0	0	**1**
Key of *bootet*	1	0	0	**1**
Key of *wudden*	0	0	1	**0**
Key of *streept*	1	1	0	**1**

	kat	*snaak*	*chaar*	Inner product
Query of *kat*	1	0	0	
Key of *kat*	1	0	0	**1**
Key of *bootet*	1	0	0	**1**
Key of *wudden*	0	0	1	**0**
Key of *streept*	1	1	0	**1**

It counts the slots where both have a 1

Entry 3

The attention mechanism

wudden

kat

vennomus

chaar

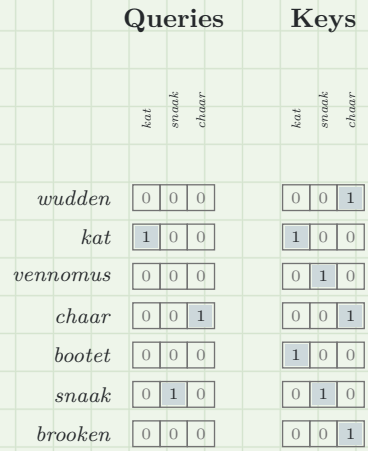
bootet

snaak

brooken

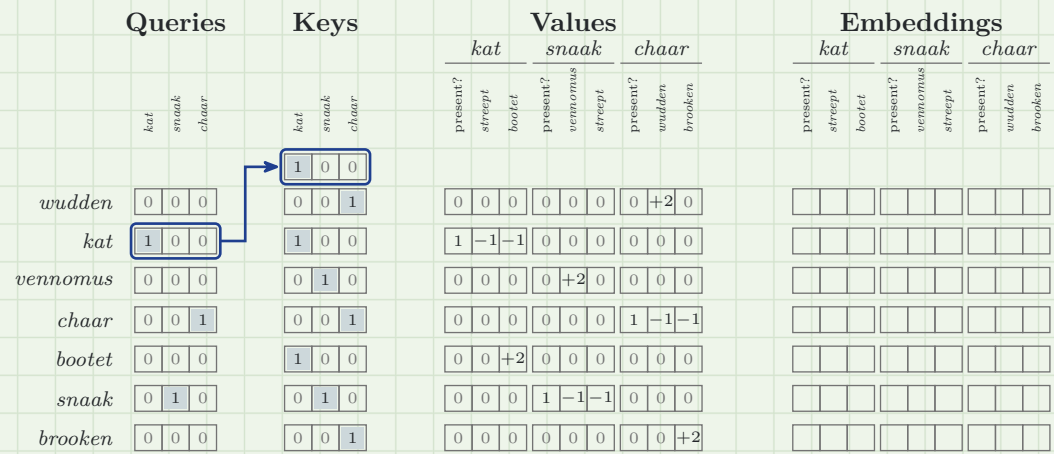
Queries

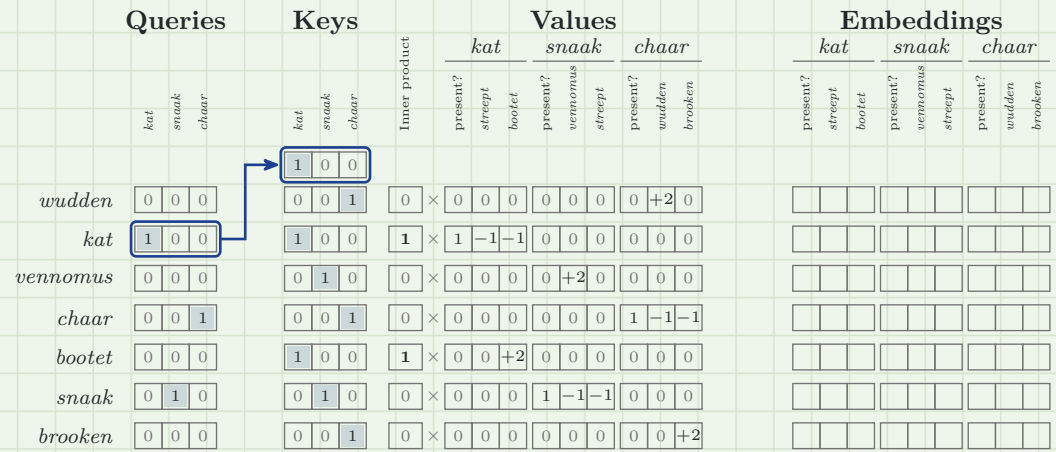
	kat	*snaak*	*chaar*
wudden	0	0	0
kat	1	0	0
vennomus	0	0	0
chaar	0	0	1
bootet	0	0	0
snaak	0	1	0
brooken	0	0	0

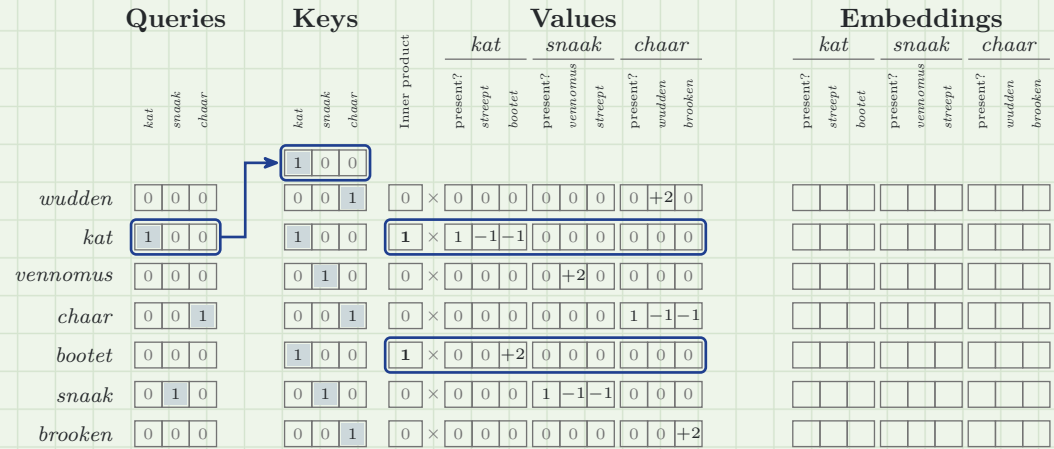

Queries
Keys
kat
snaak
chaar
kat
snaak
chaar
wudden 0 0 0 0 0 1
kat 1 0 0 1 0 0
vennomus 0 0 0 0 1 0
chaar 0 0 1 0 0 1
bootet 0 0 0 1 0 0
snaak 0 1 0 0 1 0
brooken 0 0 0 0 0 1

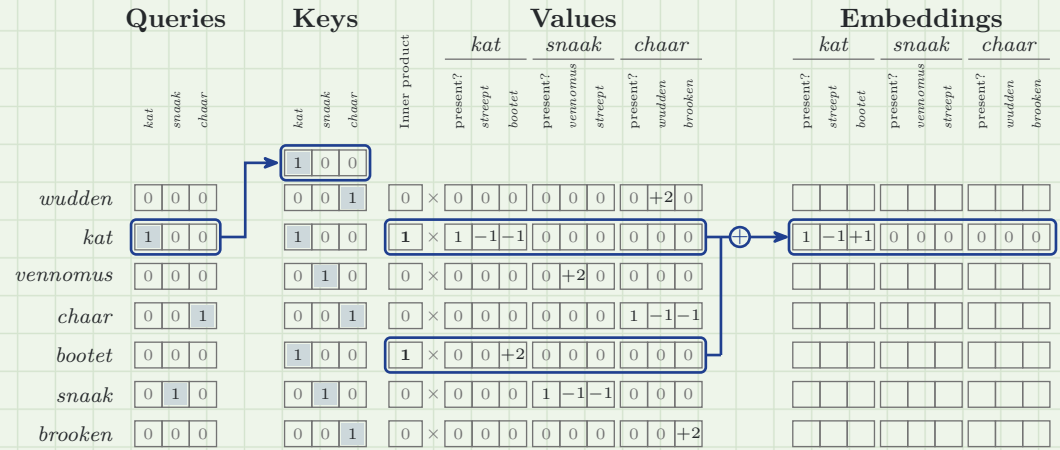
	Queries			Keys			Values								
							kat			*snaak*			*chaar*		
	kat	*snaak*	*chaar*	*kat*	*snaak*	*chaar*	present?	*streept*	*bootet*	present?	*vennomus*	*streept*	present?	*wudden*	*brooken*
wudden	0	0	0	0	0	1	0	0	0	0	0	0	0	+2	0
kat	1	0	0	1	0	0	1	−1	−1	0	0	0	0	0	0
vennomus	0	0	0	0	1	0	0	0	0	0	+2	0	0	0	0
chaar	0	0	1	0	0	1	0	0	0	0	0	0	1	−1	−1
bootet	0	0	0	1	0	0	0	0	+2	0	0	0	0	0	0
snaak	0	1	0	0	1	0	0	0	0	1	−1	−1	0	0	0
brooken	0	0	0	0	0	1	0	0	0	0	0	0	0	0	+2

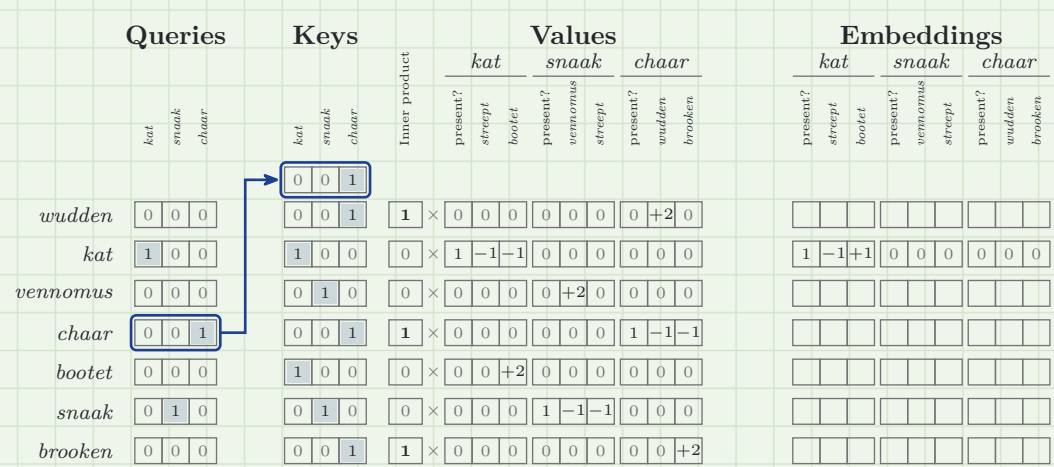
	Queries			Keys			Values									Embeddings								
							kat			*snaak*			*chaar*			*kat*			*snaak*			*chaar*		
	kat	*snaak*	*chaar*	*kat*	*snaak*	*chaar*	present?	*streept*	*bootet*	present?	*vennomus*	*streept*	present?	*wudden*	*brooken*	present?	*streept*	*bootet*	present?	*vennomus*	*streept*	present?	*wudden*	*brooken*
wudden	0	0	0	0	0	1	0	0	0	0	0	0	0	+2	0									
kat	1	0	0	1	0	0	1	−1	−1	0	0	0	0	0	0									
vennomus	0	0	0	0	1	0	0	0	0	0	+2	0	0	0	0									
chaar	0	0	1	0	0	1	0	0	0	0	0	0	1	−1	−1									
bootet	0	0	0	1	0	0	0	0	+2	0	0	0	0	0	0									
snaak	0	1	0	0	1	0	0	0	0	1	−1	−1	0	0	0									
brooken	0	0	0	0	0	1	0	0	0	0	0	0	0	0	+2									

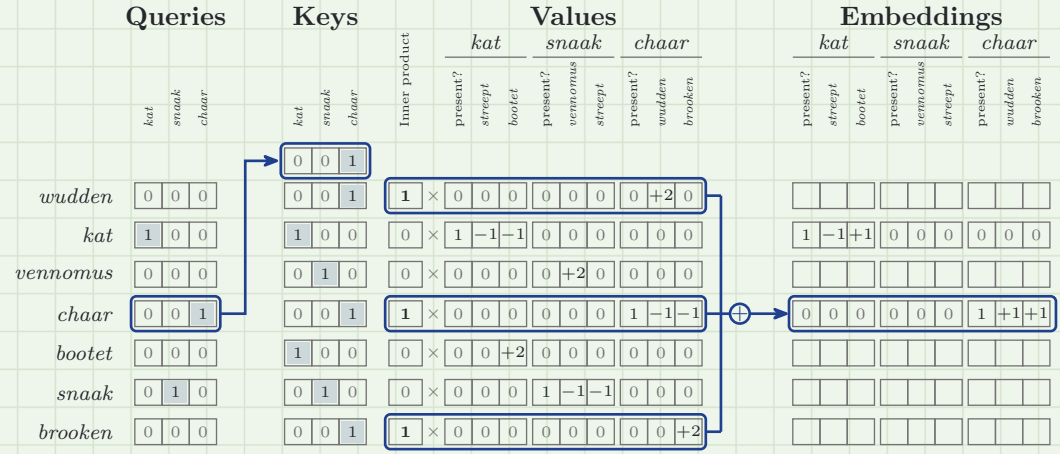

Queries
Keys
Values
Embeddings
kat
snaak
chaar
present?
streept
bootet
vennomus
wudden
brooken
wudden
kat
vennomus
chaar
bootet
snaak
brooken

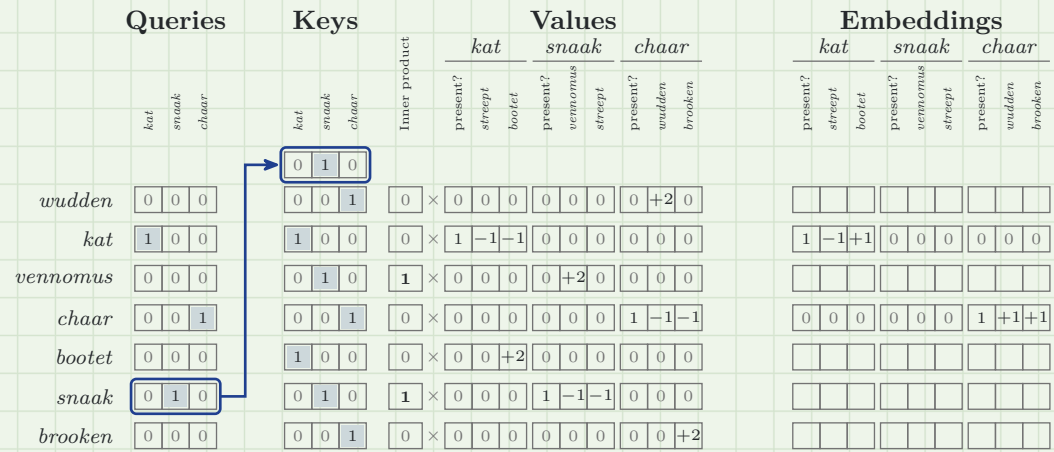

Queries
Keys
Values
Embeddings
kat
snaak
chaar
Inner product
present?
streept
bootet
vennomus
wudden
brooken
wudden
kat
vennomus
chaar
bootet
snaak
brooken

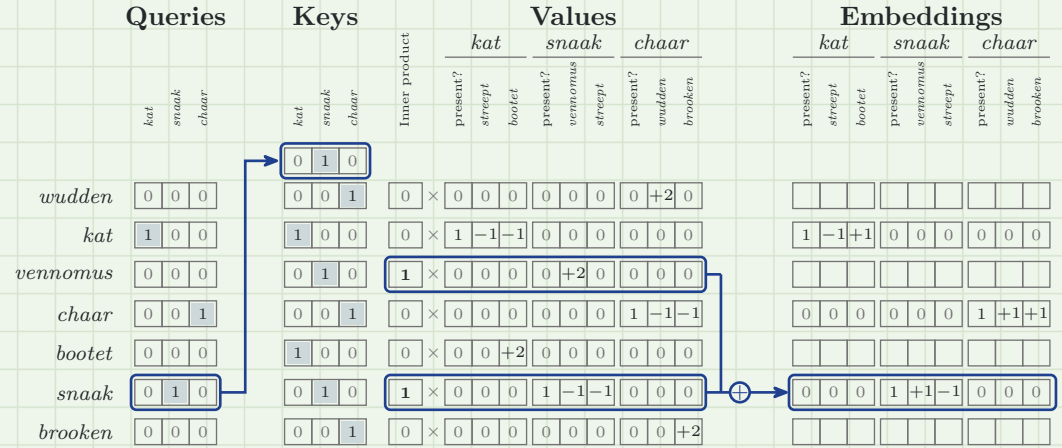

Queries
Keys
Values
Embeddings
kat
snaak
chaar
Inner product
present?
streept
bootet
vennomus
wudden
brooken
wudden
kat
vennomus
chaar
bootet
snaak
brooken

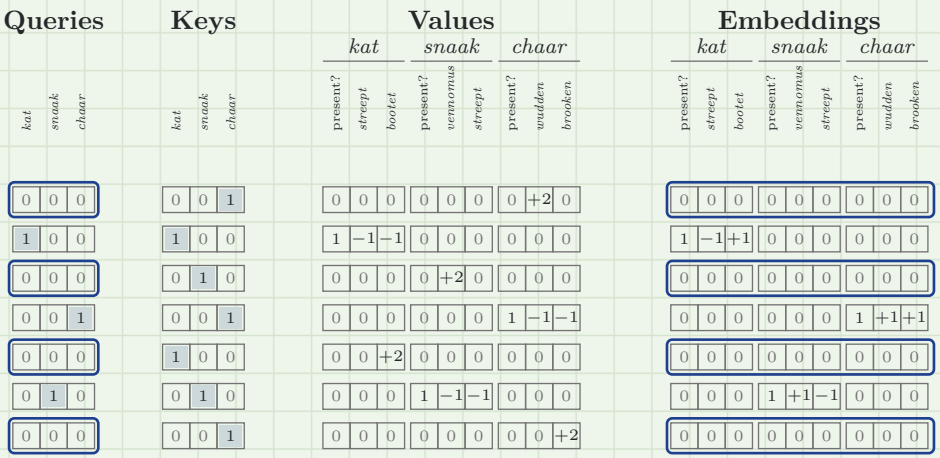

Queries
Keys
Values
Embeddings
kat
snaak
chaar
Inner product
present?
streept
bootet
vennomus
wudden
brooken
wudden
kat
vennomus
chaar
bootet
snaak
brooken

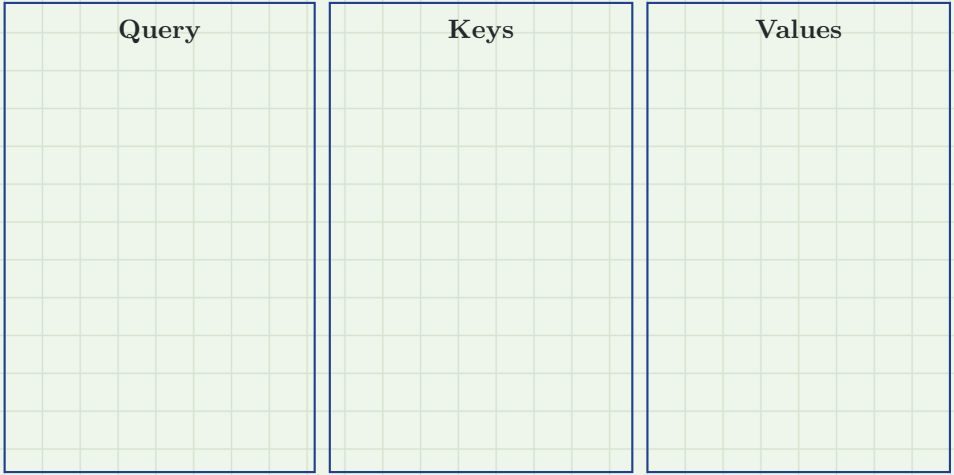

Queries
Keys
Values
Embeddings
kat
snaak
chaar
Inner product
present?
streept
bootet
vennomus
wudden
brooken
wudden
kat
vennomus
chaar
bootet
snaak
brooken


Queries
Keys
Values
Embeddings
kat
snaak
chaar
present?
streept
bootet
vennomus
wudden
brooken
Inner product
wudden
kat
vennomus
chaar
bootet
snaak
brooken


Queries
Keys
Values
Embeddings
kat
snaak
chaar
Inner product
present?
streept
bootet
vennomus
wudden
brooken
wudden
kat
vennomus
chaar
bootet
snaak
brooken

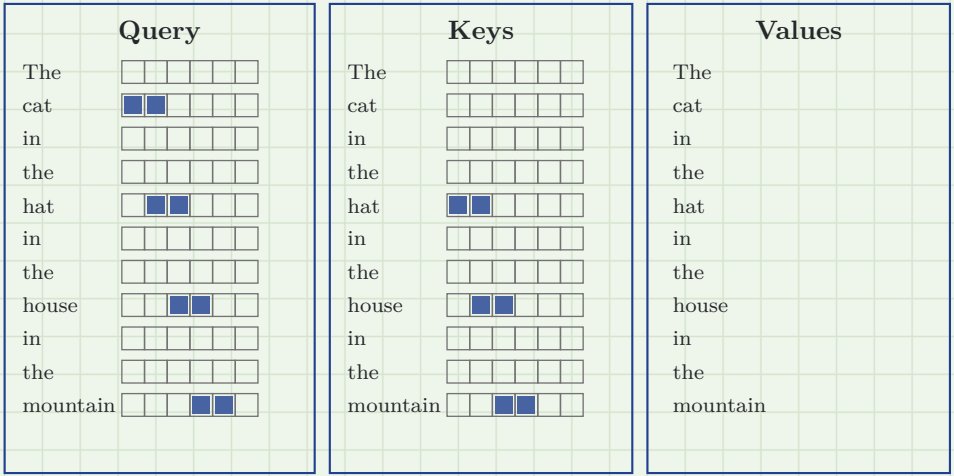

Queries
Keys
Values
Embeddings
kat
snaak
chaar
Inner product
present?
streept
bootet
vennomus
wudden
brooken
wudden
kat
vennomus
chaar
bootet
snaak
brooken

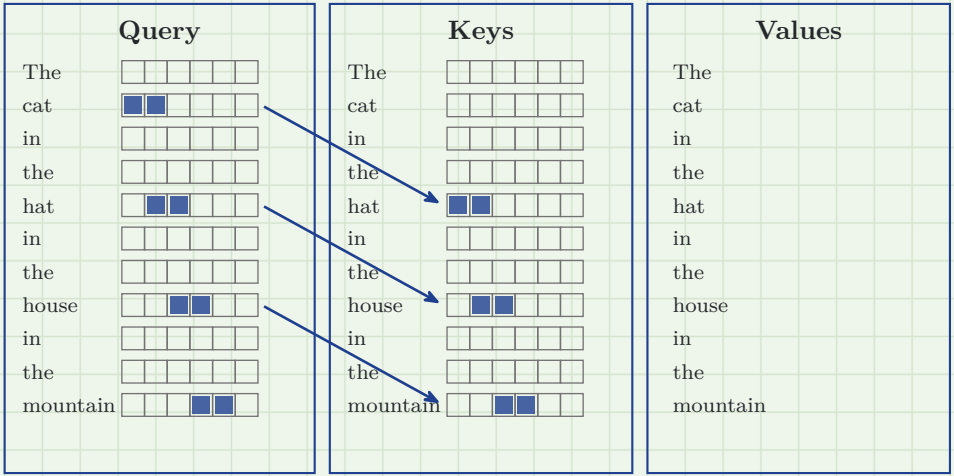

Queries
Keys
Values
Embeddings
kat
snaak
chaar
present?
streept
bootet
vennomus
wudden
brooken
wudden
kat
vennomus
chaar
bootet
snaak
brooken

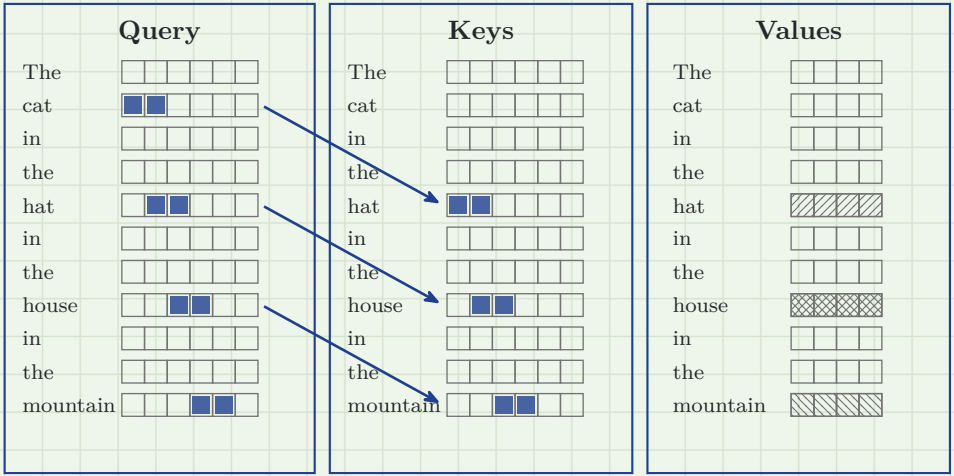

Query
Keys
Values

Query	Keys	Values
The	The	The
cat	cat	cat
in	in	in
the	the	the
hat	hat	hat
in	in	in
the	the	the
house	house	house
in	in	in
the	the	the
mountain	mountain	mountain

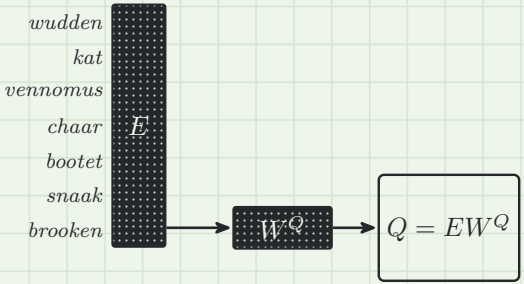
Query		Keys	Values
The		The	The
cat		cat	cat
in		in	in
the		the	the
hat		hat	hat
in		in	in
the		the	the
house		house	house
in		in	in
the		the	the
mountain		mountain	mountain

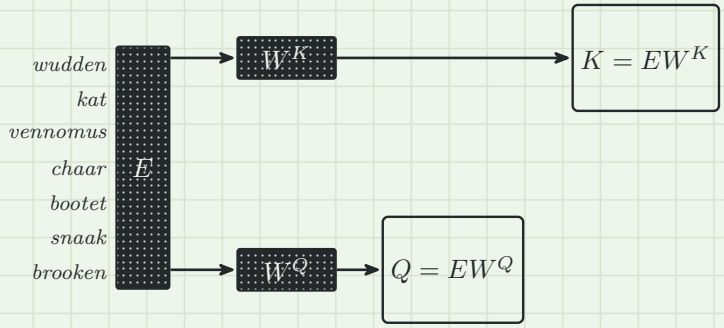

Query
The
cat
in
the
hat
in
the
house
in
the
mountain
Keys
The
cat
in
the
hat
in
the
house
in
the
mountain
Values
The
cat
in
the
hat
in
the
house
in
the
mountain

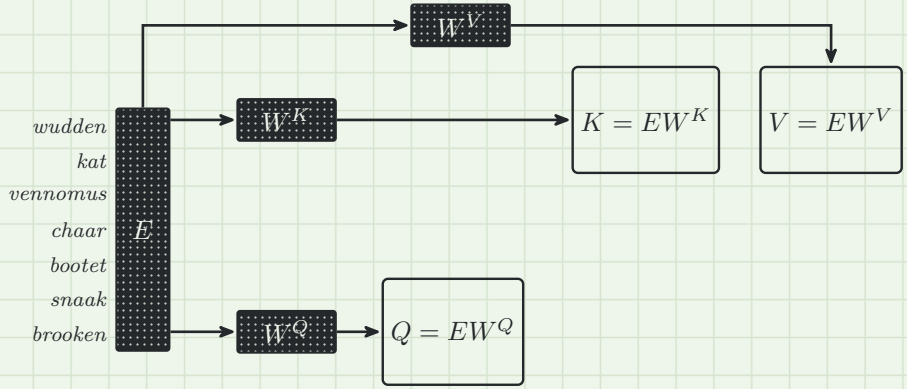

Query
The
cat
in
the
hat
in
the
house
in
the
mountain
Keys
The
cat
in
the
hat
in
the
house
in
the
mountain
Values
The
cat
in
the
hat
in
the
house
in
the
mountain

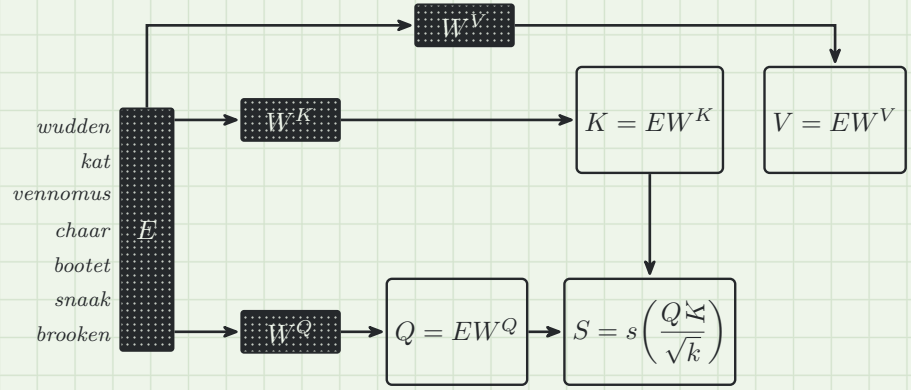

Query
Keys
Values
The
cat
in
the
hat
in
the
house
in
the
mountain

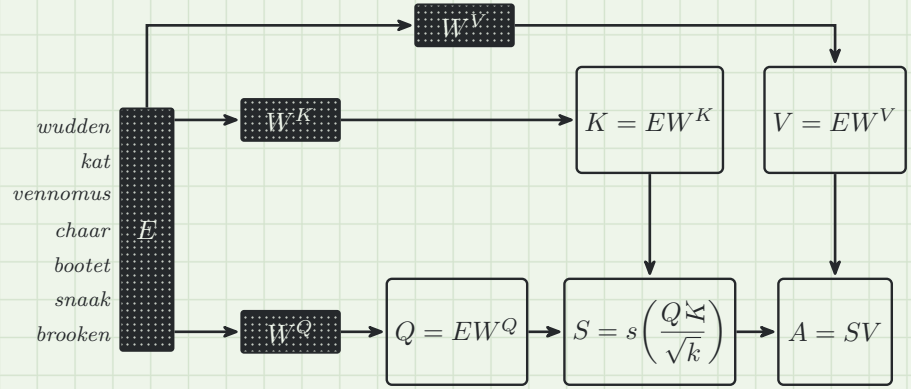

wudden
kat
vennomus
chaar
bootet
snaak
brooken
E

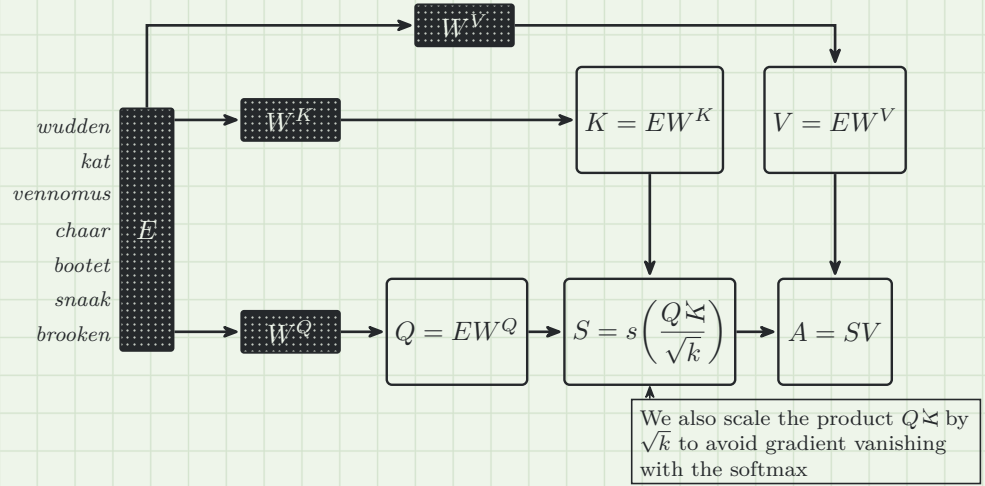

wudden
kat
vennomus
chaar
bootet
snaak
brooken
E
W^Q
$Q = EW^Q$

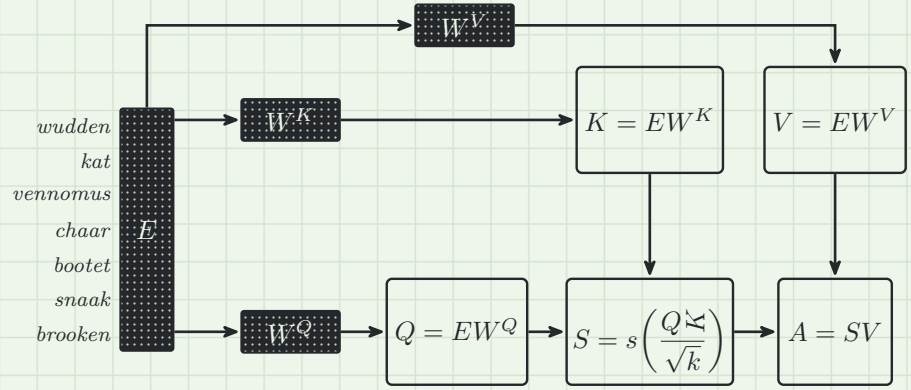

wudden
kat
vennomus
chaar
bootet
snaak
brooken
E
W^K
W^Q
$K = EW^K$
$Q = EW^Q$

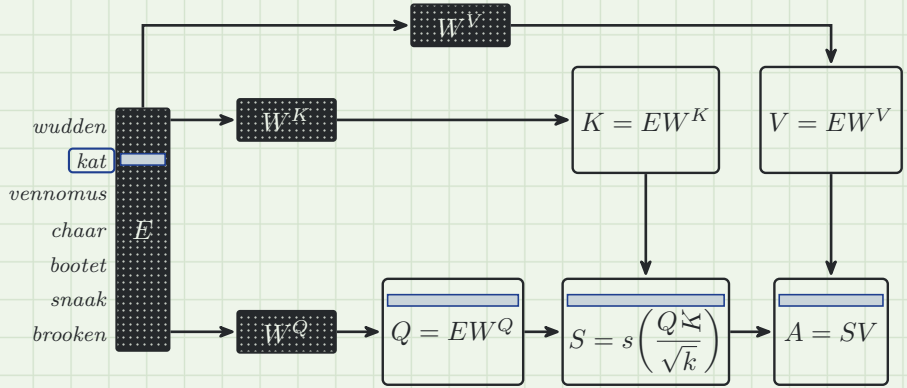

W^V
W^K
$K = EW^K$
$V = EW^V$
wudden
kat
vennomus
chaar
bootet
snaak
brooken
E
W^Q
$Q = EW^Q$


wudden
kat
vennomus
chaar
bootet
snaak
brooken
E
W^V
W^K
W^Q
$K = EW^K$
$V = EW^V$
$Q = EW^Q$
$S = s\left(\frac{Q\overleftarrow{K}}{\sqrt{k}}\right)$


wudden
kat
vennomus
chaar
bootet
snaak
brooken
E
W^V
W^K
W^Q
$K = EW^K$
$V = EW^V$
$Q = EW^Q$
$S = s\left(\frac{Q\overleftarrow{K}}{\sqrt{k}}\right)$
$A = SV$


wudden
kat
vennomus
chaar
bootet
snaak
brooken
E
W^V
W^K
W^Q
$K = EW^K$
$V = EW^V$
$Q = EW^Q$
$S = s\left(\frac{Q\overleftarrow{K}}{\sqrt{k}}\right)$
$A = SV$
We also scale the product $Q\overleftarrow{K}$ by $\sqrt{k}$ to avoid gradient vanishing with the softmax


wudden
kat
vennomus
chaar
bootet
snaak
brooken
E
W^V
W^K
W^Q
$K = EW^K$
$V = EW^V$
$Q = EW^Q$
$S = s\left(\frac{QK}{\sqrt{k}}\right)$
$A = SV$


W^V
wudden
kat
vennomus
chaar
bootet
snaak
brooken
E
W^K
$K = EW^K$
$V = EW^V$
W^Q
$Q = EW^Q$
$S = s\left(\frac{QK}{\sqrt{k}}\right)$
$A = SV$

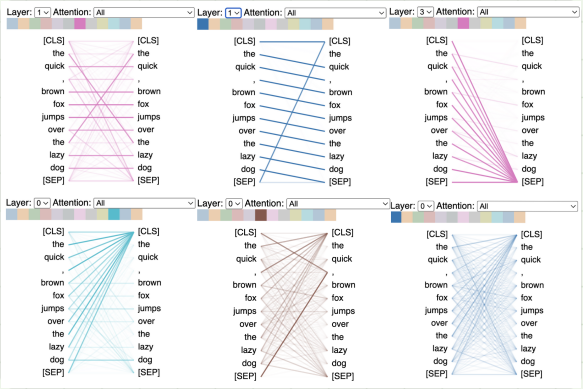
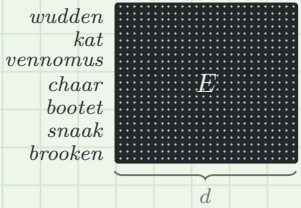
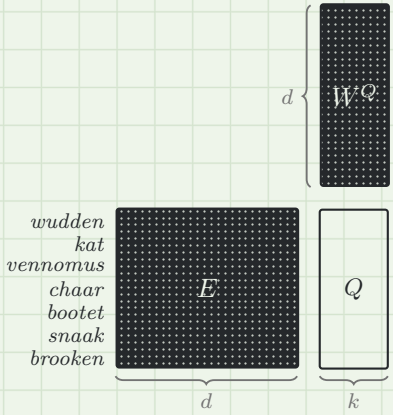
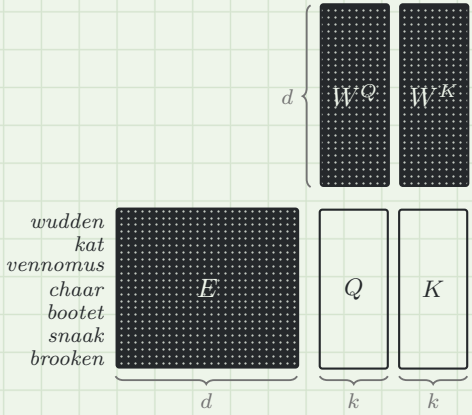


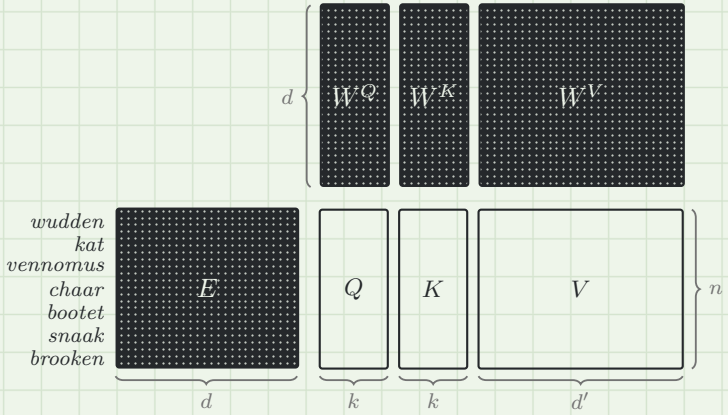
Image from: https://www.comet.com/site/blog/explainable-ai-for-transformers/

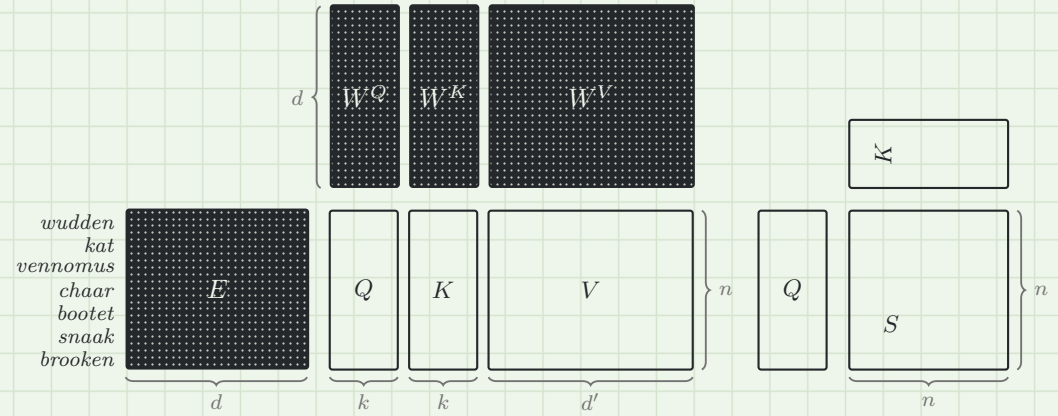
The shapes behind one attention map

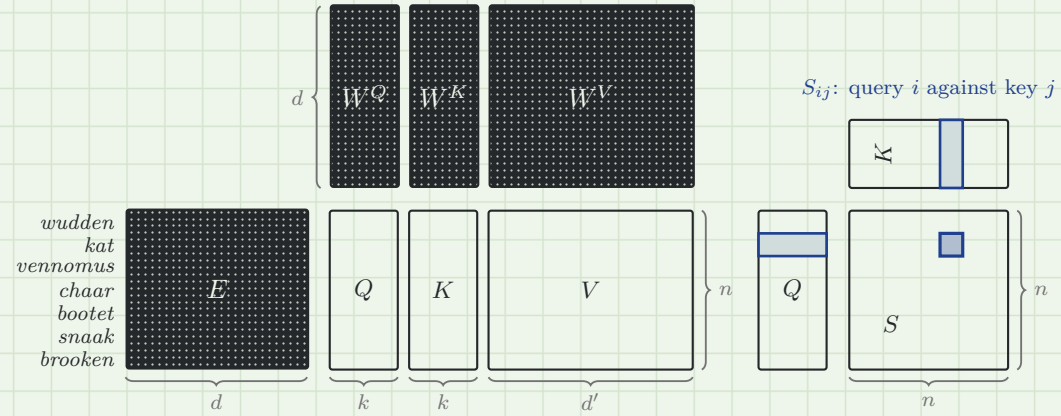


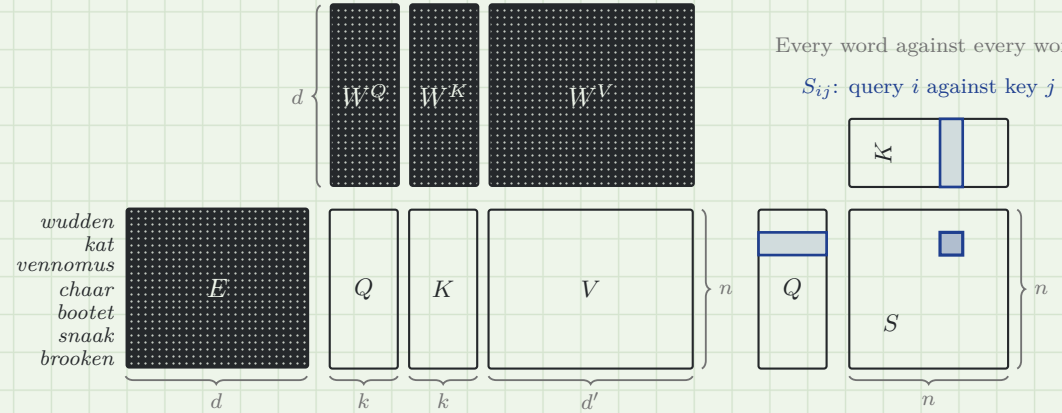

d
W^Q
wudden
kat
vennomus
chaar
bootet
snaak
brooken
E
Q
d
k


d
W^Q
W^K
wudden
kat
vennomus
chaar
bootet
snaak
brooken
E
Q
K
d
k
k


d
W^Q
W^K
W^V
wudden
kat
vennomus
chaar
bootet
snaak
brooken
E
Q
K
V
n
d
k
k
d′


d
W^Q
W^K
W^V
K
wudden
kat
vennomus
chaar
bootet
snaak
brooken
E
Q
K
V
n
Q
S
n
d
k
k
d′
n


W^Q
W^K
W^V
d
S_{ij}: query i against key j
K
wudden
kat
vennomus
chaar
bootet
snaak
brooken
E
Q
K
V
n
Q
S
n
d
k
k
d'
n


d
W^Q
W^K
W^V
Every word against every word
S_{ij}: query i against key j
K
wudden
kat
vennomus
chaar
bootet
snaak
brooken
E
Q
K
V
n
Q
S
n
d
k
k
d′
n

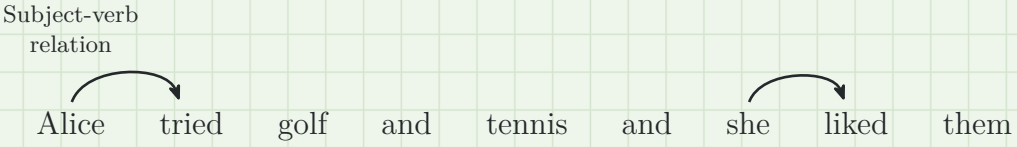
Entry 4

Multi-headed attention

Remember that we need multiple attention maps.

Alice	tried	golf	and	tennis	and	she	liked	them

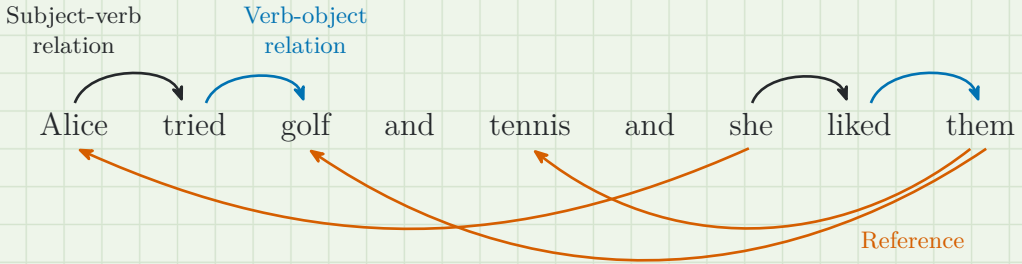
Remember that we need multiple attention maps.

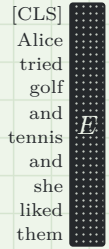


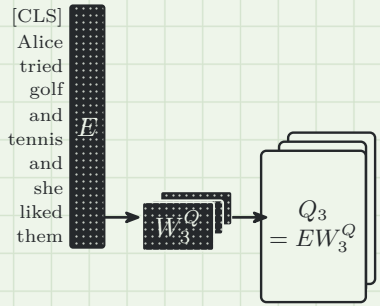
Remember that we need multiple attention maps.

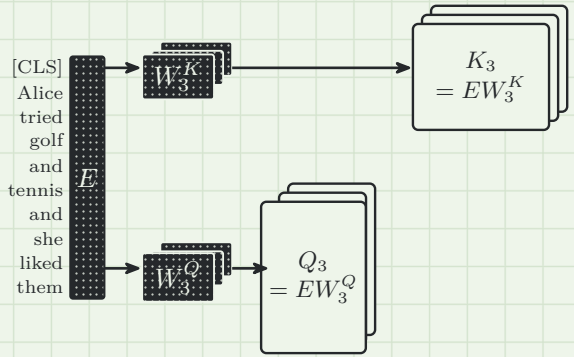


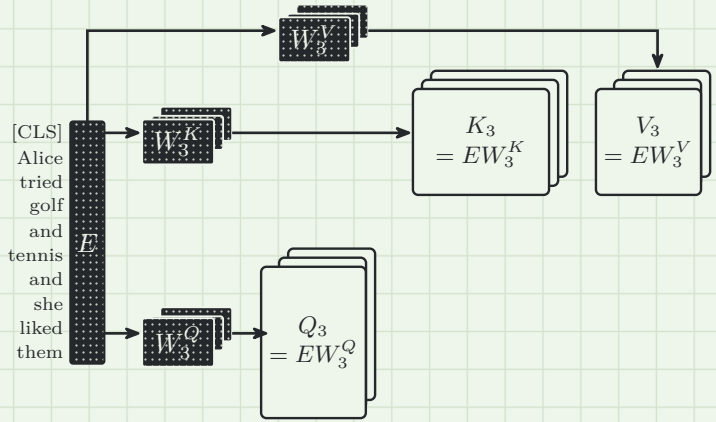
Remember that we need multiple attention maps.

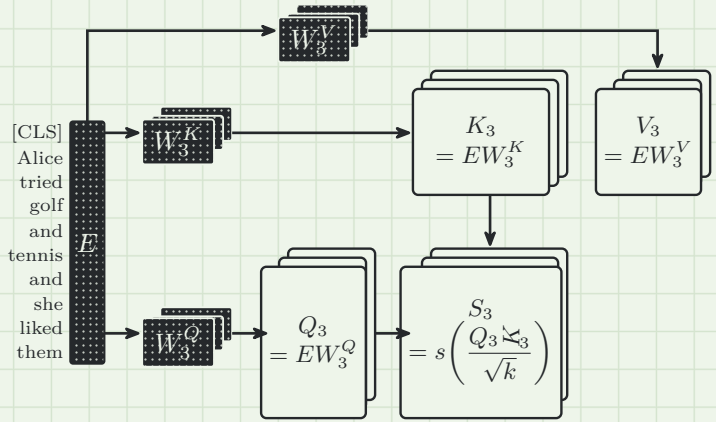


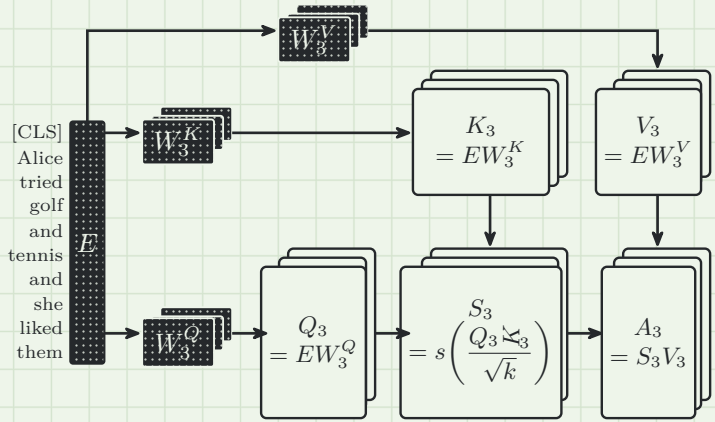

[CLS]
Alice
tried
golf
and
tennis
and
she
liked
them
E

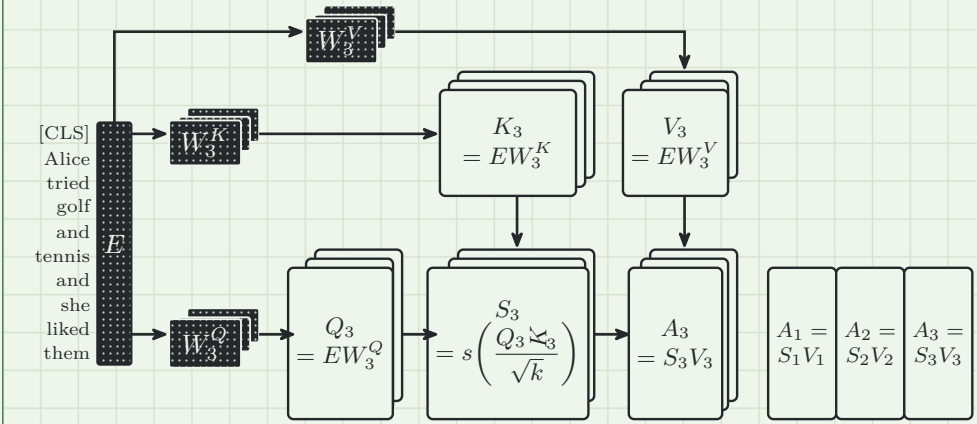

[CLS]
Alice
tried
golf
and
tennis
and
she
liked
them
E
W_3^Q
Q_3
$= EW_3^Q$

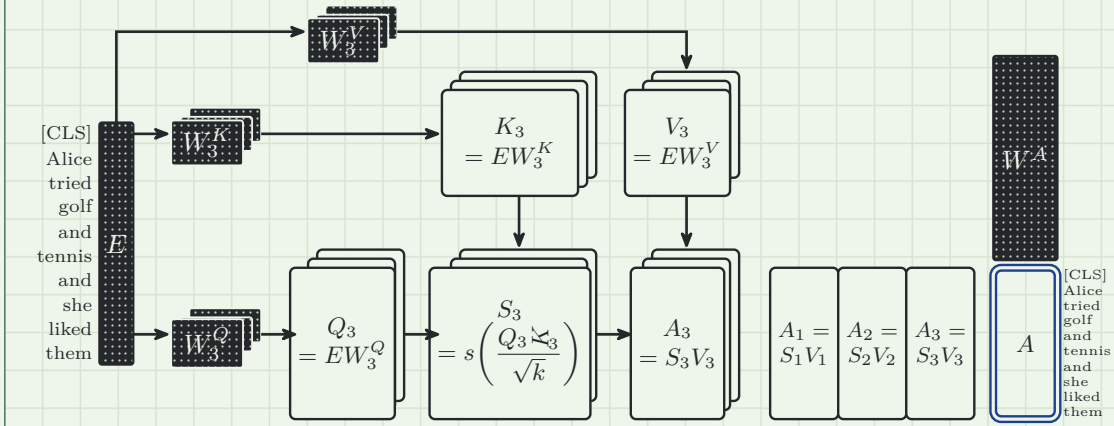

[CLS]
Alice
tried
golf
and
tennis
and
she
liked
them
E
W_3^K
W_3^Q
$K_3 = EW_3^K$
$Q_3 = EW_3^Q$


[CLS]
Alice
tried
golf
and
tennis
and
she
liked
them
E
W_3^V
W_3^K
W_3^Q
$K_3 = EW_3^K$
$V_3 = EW_3^V$
$Q_3 = EW_3^Q$


[CLS]
Alice
tried
golf
and
tennis
and
she
liked
them
E
W_3^Q
W_3^K
W_3^V
$Q_3 = EW_3^Q$
$K_3 = EW_3^K$
$V_3 = EW_3^V$
$S_3 = s\left(\frac{Q_3 K_3}{\sqrt{k}}\right)$


[CLS]
Alice
tried
golf
and
tennis
and
she
liked
them
E
W_3^V
W_3^K
W_3^Q
$K_3 = EW_3^K$
$V_3 = EW_3^V$
$Q_3 = EW_3^Q$
$S_3 = s\left(\frac{Q_3 K_3}{\sqrt{k}}\right)$
$A_3 = S_3 V_3$


[CLS]
Alice
tried
golf
and
tennis
and
she
liked
them
E
W_3^V
W_3^K
W_3^Q
$K_3 = EW_3^K$
$V_3 = EW_3^V$
$Q_3 = EW_3^Q$
$S_3 = s\left(\frac{Q_3 K_3}{\sqrt{k}}\right)$
$A_3 = S_3 V_3$
$A_1 = S_1 V_1$
$A_2 = S_2 V_2$
$A_3 = S_3 V_3$


[CLS] Alice tried golf and tennis and she liked them
E
W_3^V
W_3^K
W_3^Q
$K_3 = EW_3^K$
$V_3 = EW_3^V$
$Q_3 = EW_3^Q$
$S_3 = s\left(\frac{Q_3 K_3}{\sqrt{k}}\right)$
$A_3 = S_3 V_3$
$A_1 = S_1 V_1$
$A_2 = S_2 V_2$
$A_3 = S_3 V_3$
W^A
A
[CLS] Alice tried golf and tennis and she liked them

Entry 5

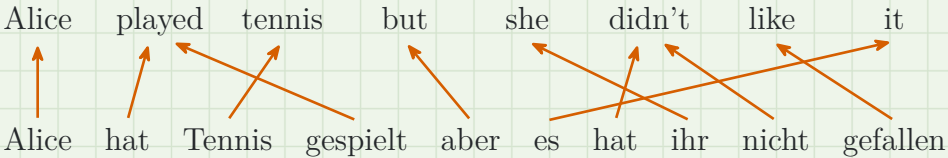
Cross-attention

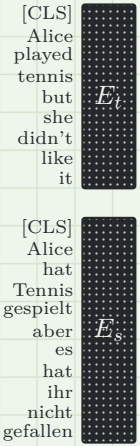
- Consider the problem of translating from English to German.

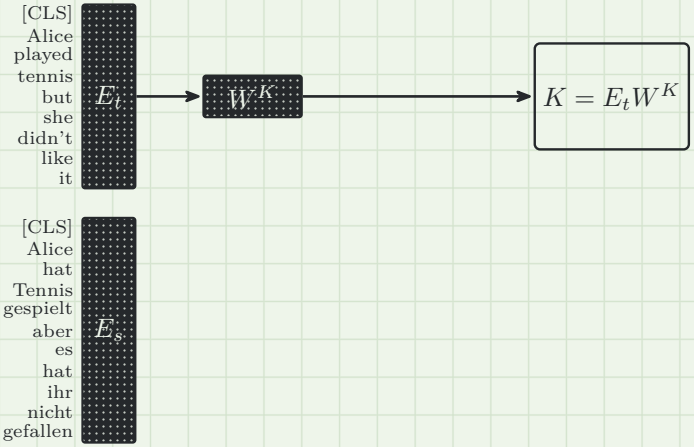
Alice played tennis but she didn't like it

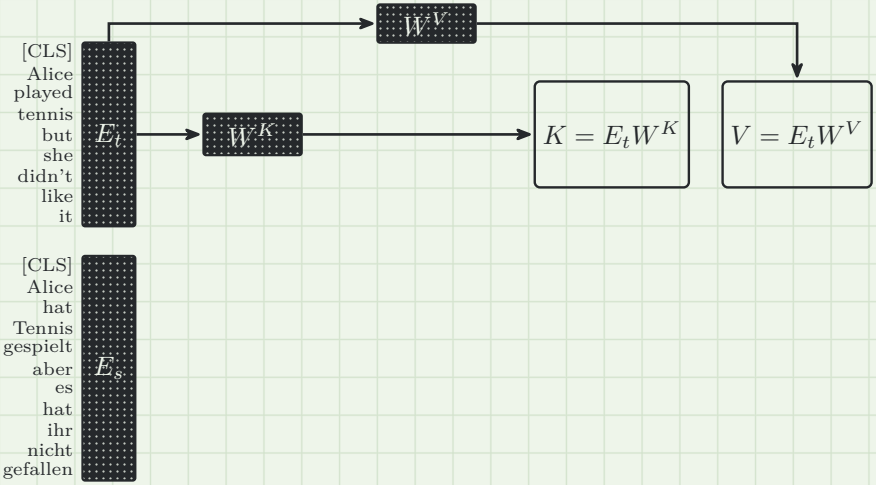
Alice hat Tennis gespielt aber es hat ihr nicht gefallen

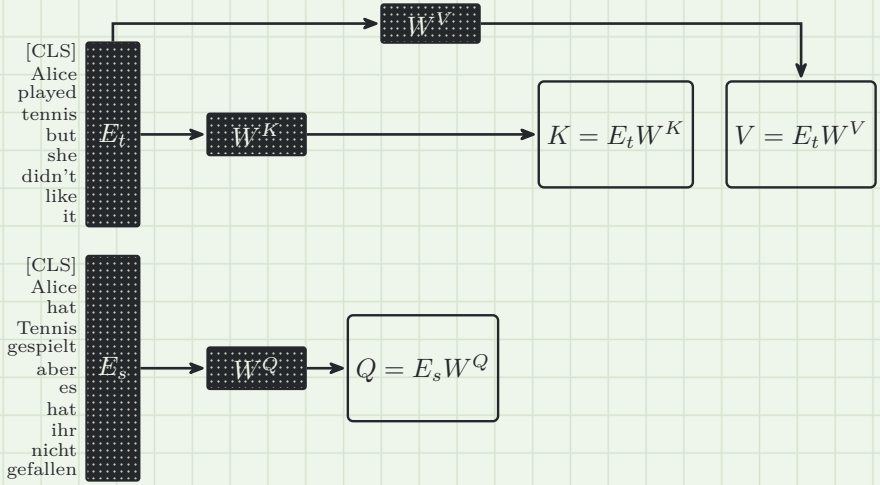
- Consider the problem of translating from English to German.
- A different attention mechanism is needed.

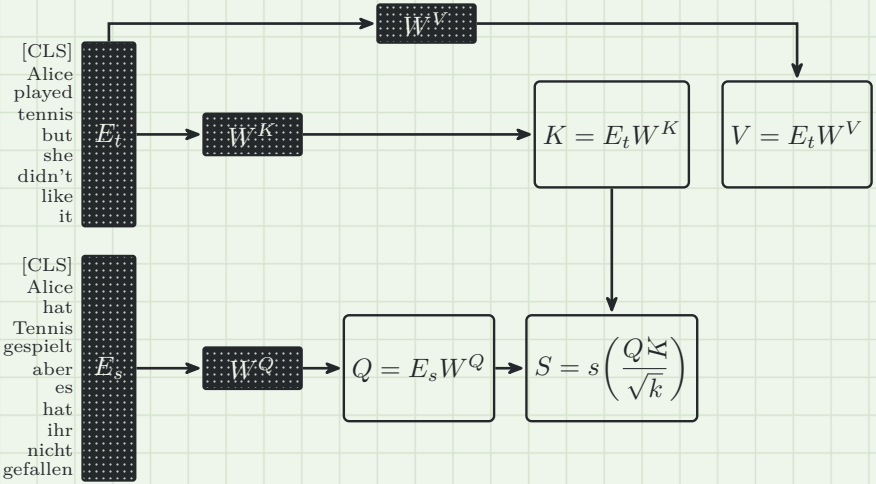


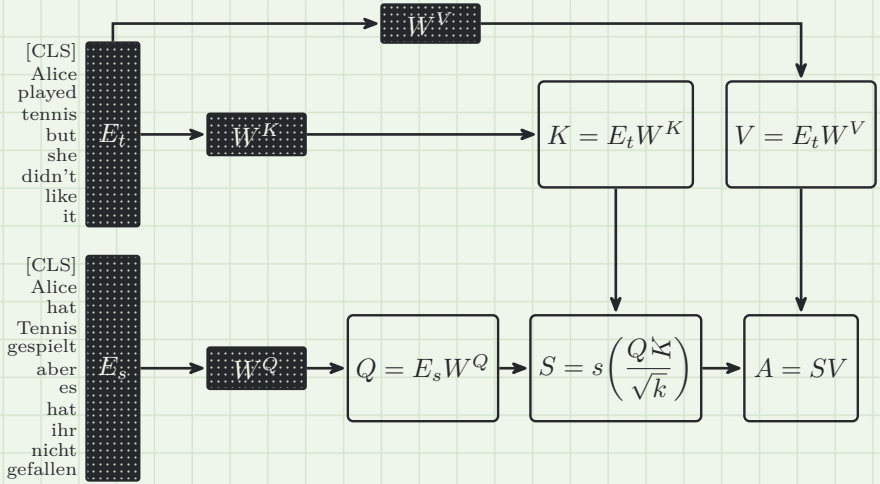

[CLS]
Alice
played
tennis
but
she
didn't
like
it
E_t
[CLS]
Alice
hat
Tennis
gespielt
aber
es
hat
ihr
nicht
gefallen
E_s


[CLS] Alice played tennis but she didn't like it
E_t
W^K
$K = E_t W^K$
[CLS] Alice hat Tennis gespielt aber es hat ihr nicht gefallen
E_s


[CLS] Alice played tennis but she didn't like it
E_t
W^V
W^K
$K = E_t W^K$
$V = E_t W^V$
[CLS] Alice hat Tennis gespielt aber es hat ihr nicht gefallen
E_s


[CLS] Alice played tennis but she didn't like it
E_t
W^V
W^K
$K = E_t W^K$
$V = E_t W^V$
[CLS] Alice hat Tennis gespielt aber es hat ihr nicht gefallen
E_s
W^Q
$Q = E_s W^Q$


[CLS] Alice played tennis but she didn't like it
E_t
W^V
W^K
$K = E_t W^K$
$V = E_t W^V$
[CLS] Alice hat Tennis gespielt aber es hat ihr nicht gefallen
E_s
W^Q
$Q = E_s W^Q$
$S = s\left(\frac{Q\overset{\curvearrowright}{K}}{\sqrt{k}}\right)$


[CLS] Alice played tennis but she didn't like it
E_t
W^V
W^K
$K = E_t W^K$
$V = E_t W^V$
[CLS] Alice hat Tennis gespielt aber es hat ihr nicht gefallen
E_s
W^Q
$Q = E_s W^Q$
$S = s\left(\frac{Q\overline{K}}{\sqrt{k}}\right)$
$A = SV$

Entry 6

Masked self-attention

- Consider the task of generating text for answering questions.

Why did the bicycle fall over

- Consider the task of generating text for answering questions.

Why did the bicycle fall over

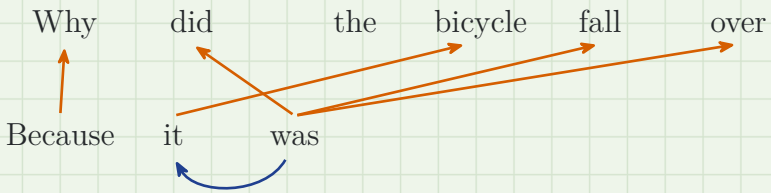


Because

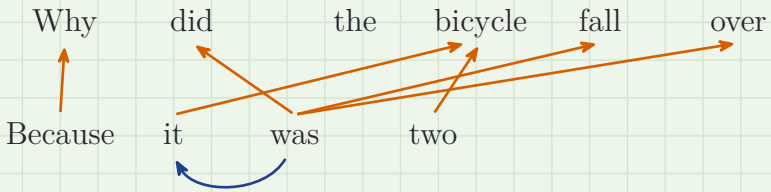
- Consider the task of generating text for answering questions.



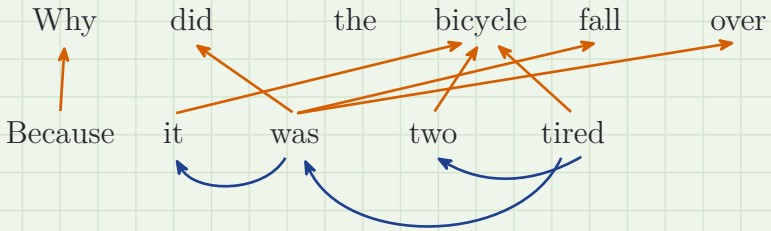
- Consider the task of generating text for answering questions.



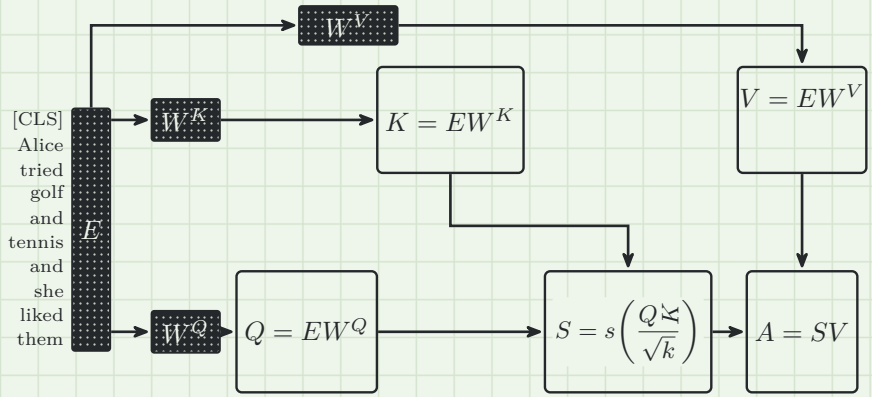
- Consider the task of generating text for answering questions.

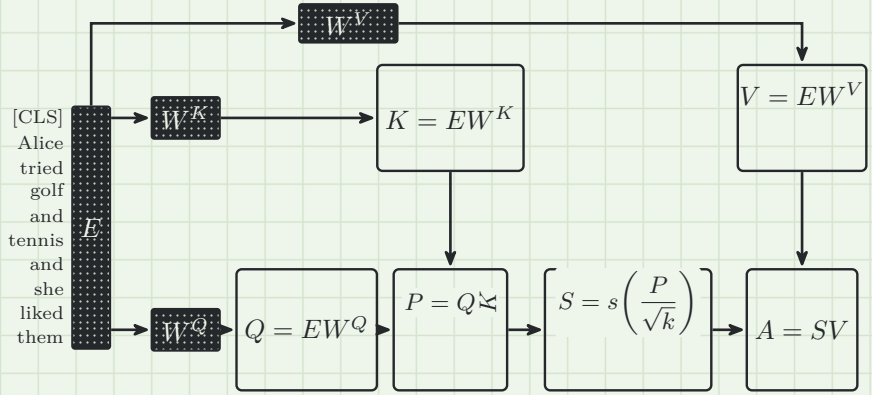


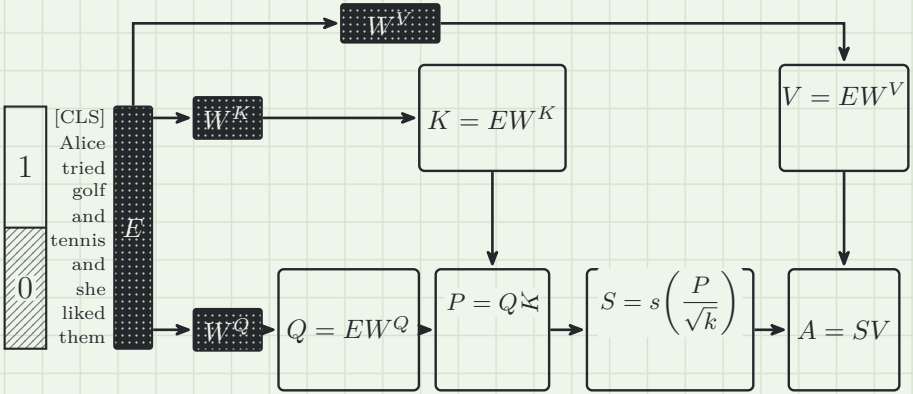
- Consider the task of generating text for answering questions.

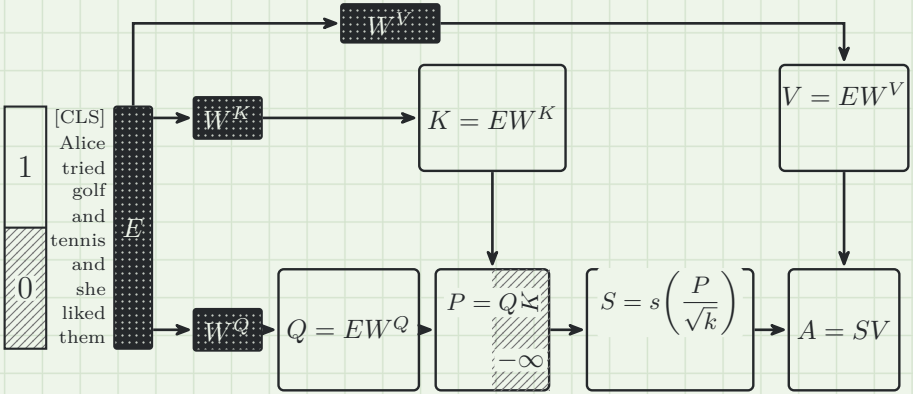


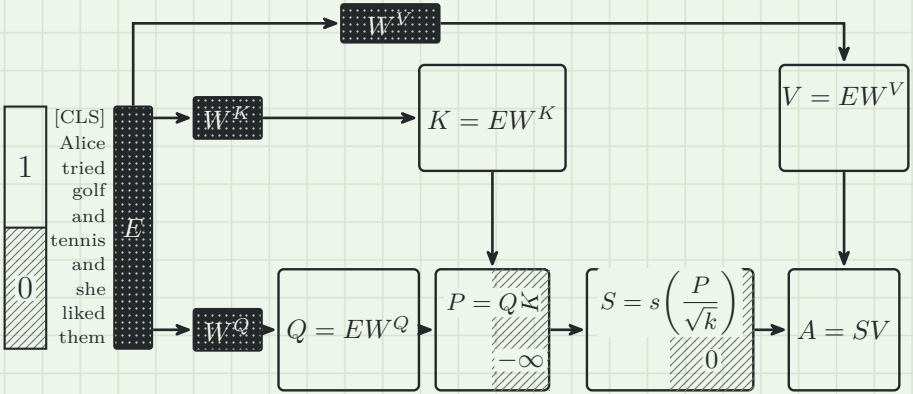
For generating sentences, we need an attention mechanism that only looks at what has been produced so far.

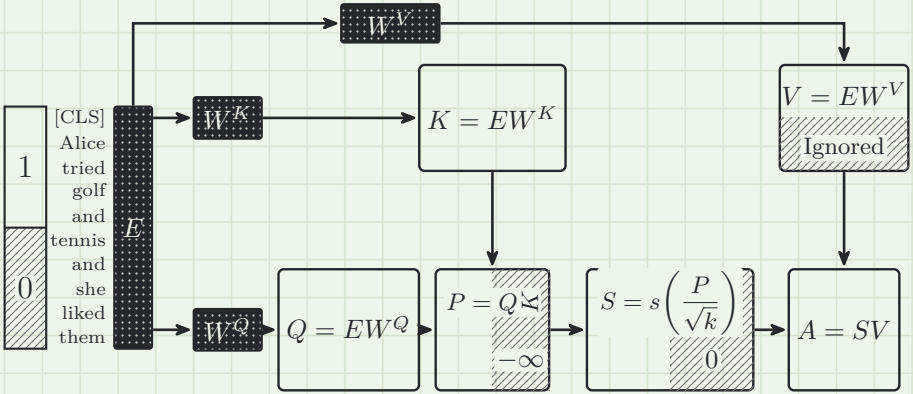

[CLS]
Alice
tried
golf
and
tennis
and
she
liked
them
E
W^V
W^K
W^Q
$K = EW^K$
$V = EW^V$
$Q = EW^Q$
$S = s\left(\frac{Q\overline{K}}{\sqrt{k}}\right)$
$A = SV$


[CLS]
Alice
tried
golf
and
tennis
and
she
liked
them
E
W^V
W^K
W^Q
$K = EW^K$
$V = EW^V$
$Q = EW^Q$
$P = QK$
$S = s\left(\frac{P}{\sqrt{k}}\right)$
$A = SV$

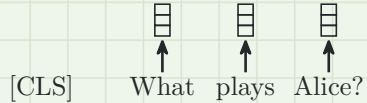

W^V
W^K
$K = EW^K$
$V = EW^V$
1
0
[CLS]
Alice
tried
golf
and
tennis
and
she
liked
them
E
W^Q
$Q = EW^Q$
$P = QK$
$S = s\left(\frac{P}{\sqrt{k}}\right)$
$A = SV$

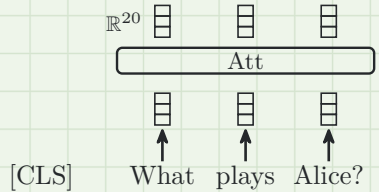

W^V
W^K
$K = EW^K$
$V = EW^V$
1
0
[CLS]
Alice
tried
golf
and
tennis
and
she
liked
them
E
W^Q
$Q = EW^Q$
$P = QK$
$-\infty$
$S = s\left(\frac{P}{\sqrt{k}}\right)$
$A = SV$

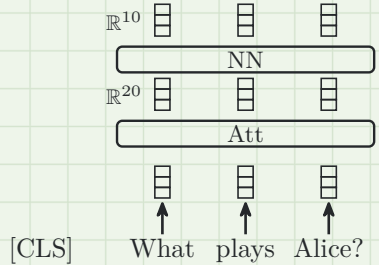

W^V
W^K
$K = EW^K$
$V = EW^V$
1
0
[CLS]
Alice
tried
golf
and
tennis
and
she
liked
them
E
W^Q
$Q = EW^Q$
$P = QK$
$-\infty$
$S = s\left(\frac{P}{\sqrt{k}}\right)$
0
$A = SV$

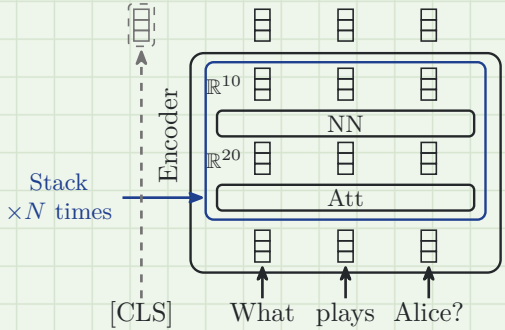


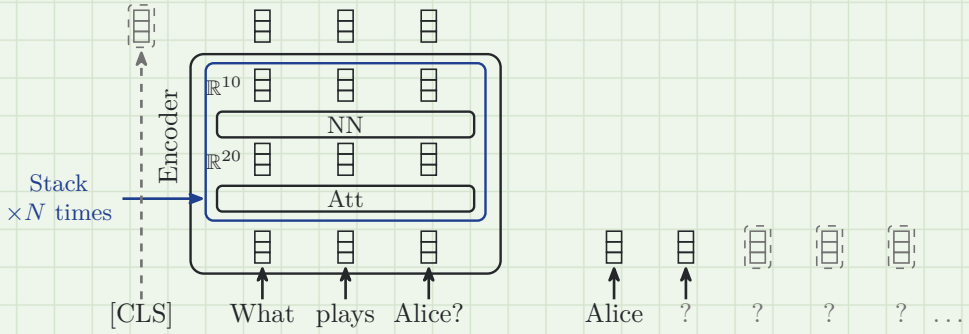
Note how the final embeddings depend only on the embeddings of the unmasked tokens!

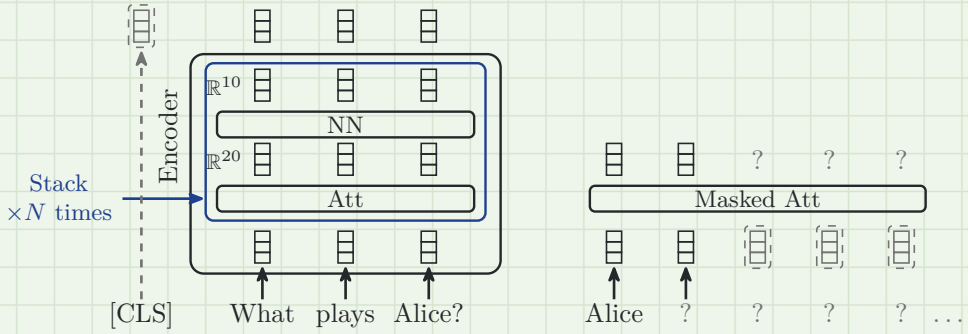

[CLS]
What
plays
Alice?

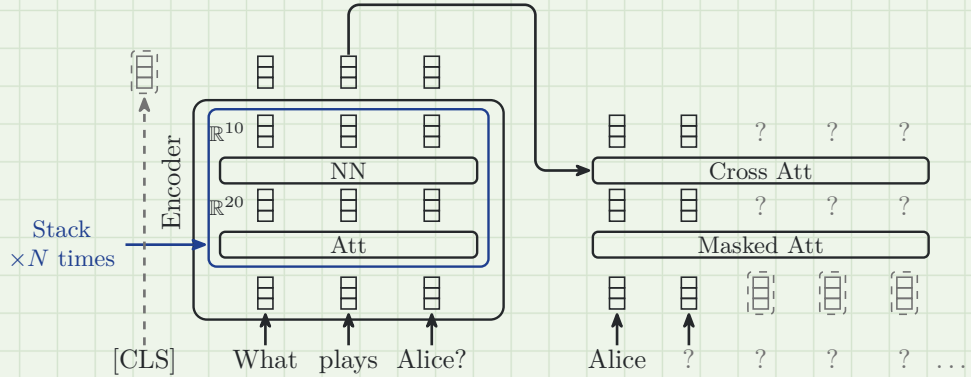

$\mathbb{R}^{20}$
Att
[CLS]
What
plays
Alice?

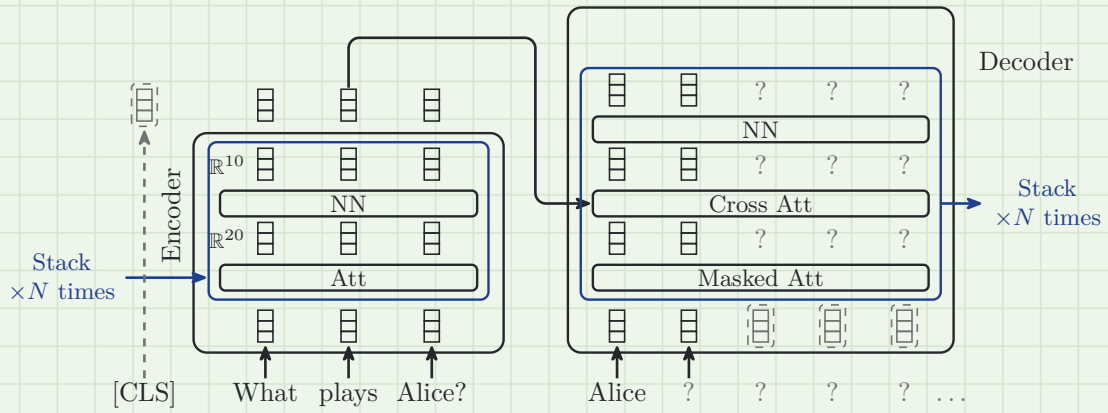

$\mathbb{R}^{10}$
NN
$\mathbb{R}^{20}$
Att
[CLS]
What
plays
Alice?

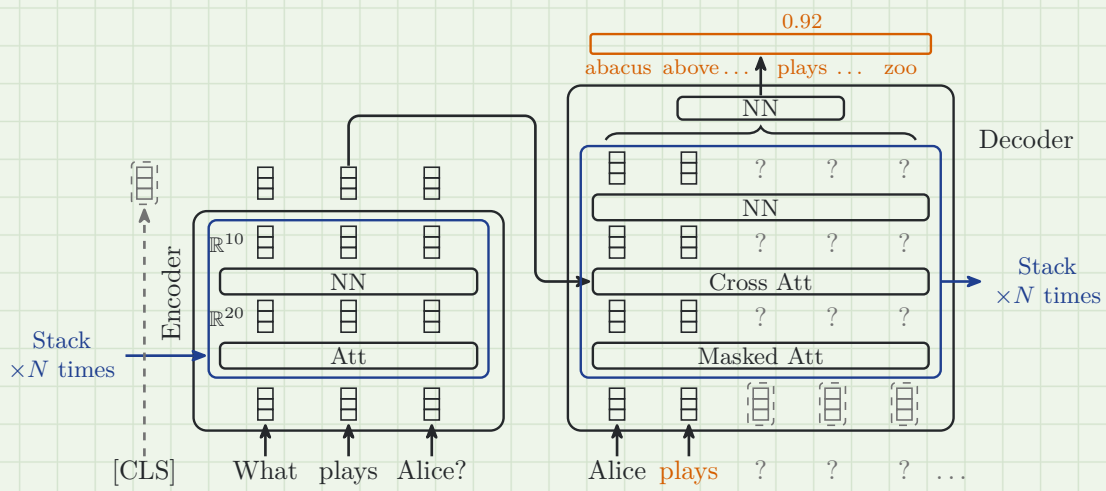

Stack
$\times N$ times
Encoder
$\mathbb{R}^{10}$
NN
$\mathbb{R}^{20}$
Att
[CLS]
What
plays
Alice?


Encoder
$\mathbb{R}^{10}$
NN
$\mathbb{R}^{20}$
Att
Stack
$\times N$ times
[CLS]
What
plays
Alice?
Alice
?
?
?
?
...


Encoder
$\mathbb{R}^{10}$
NN
$\mathbb{R}^{20}$
Att
Stack
$\times N$ times
[CLS]
What
plays
Alice?
Masked Att
Alice
? ? ? ? ? ? ? ? ? ...


Encoder
$\mathbb{R}^{10}$
NN
$\mathbb{R}^{20}$
Att
Stack
$\times N$ times
Cross Att
Masked Att
?
[CLS]
What
plays
Alice?
Alice
...


Decoder
Encoder
Stack
×N times
Stack
×N times
ℝ10
ℝ20
NN
Att
NN
Cross Att
Masked Att
?
[CLS]
What
plays
Alice?
Alice
. . .


0.92
abacus above . . . plays . . . zoo
NN
Decoder
? ? ?
NN
? ? ?
Cross Att
? ? ?
Masked Att
Stack
×N times
Encoder
ℝ10
NN
ℝ20
Att
Stack
×N times
[CLS]
What plays Alice?
Alice plays ? ? ? . . .

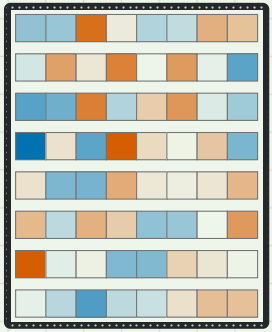
Entry 7

Training a transformer

One row of numbers per word

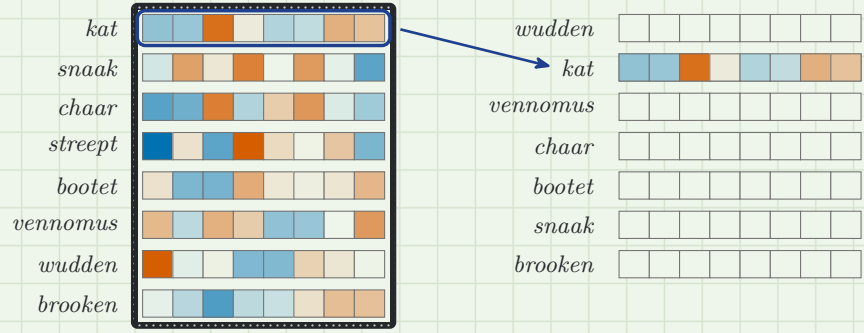


One row of numbers per word



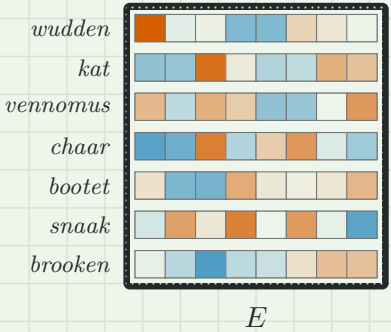
wudden
kat
vennomus
chaar
bootet
snaak
brooken

One row of numbers per word



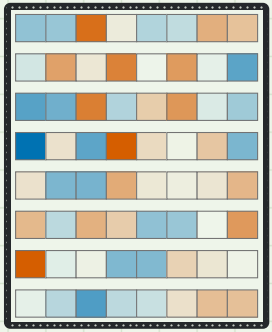
One row of numbers per word

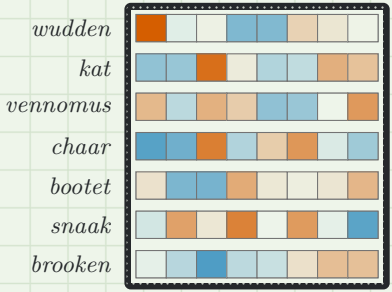




E

One row of numbers per word





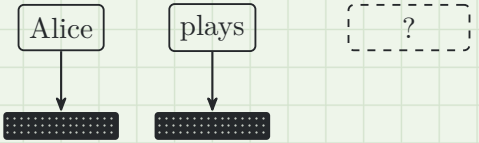
E

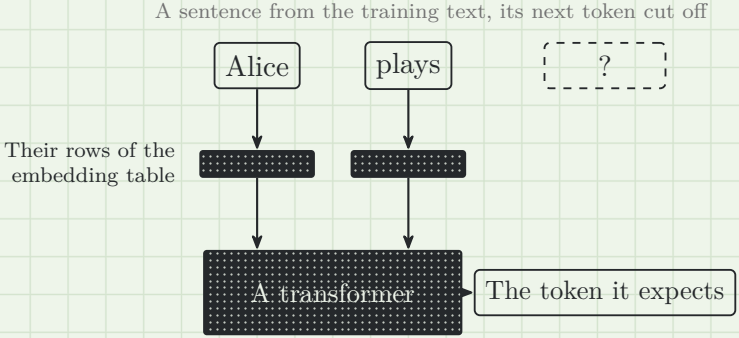
These numbers are weights too, like W^Q, W^K and W^V. Training guesses them

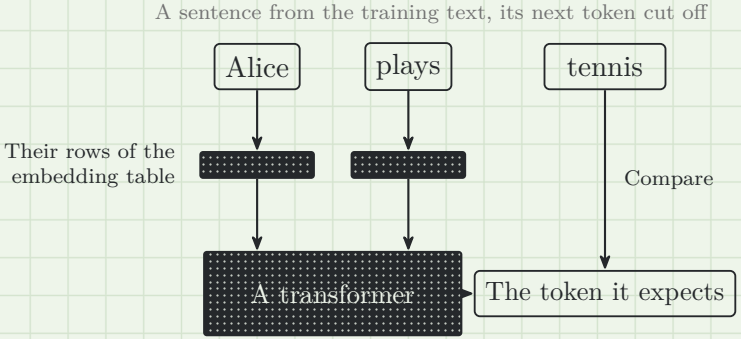
A sentence from the training text, its next token cut off

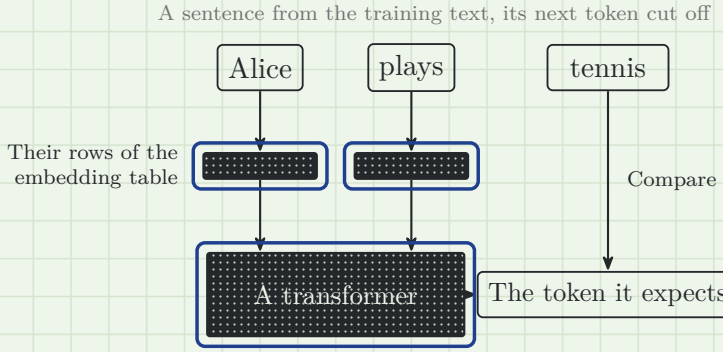
Alice plays ?

A sentence from the training text, its next token cut off




A sentence from the training text, its next token cut off
Alice
plays
?
Their rows of the embedding table
A transformer
The token it expects

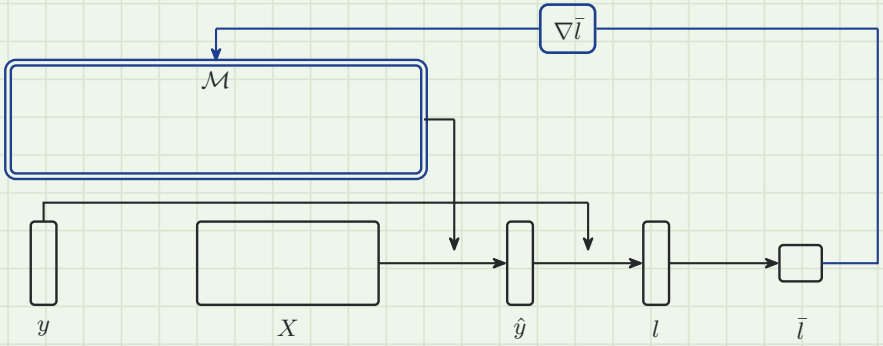

A sentence from the training text, its next token cut off
Alice
plays
tennis
Their rows of the embedding table
Compare
A transformer
The token it expects



Not tennis? Every outlined weight moves a little toward it, both rows included

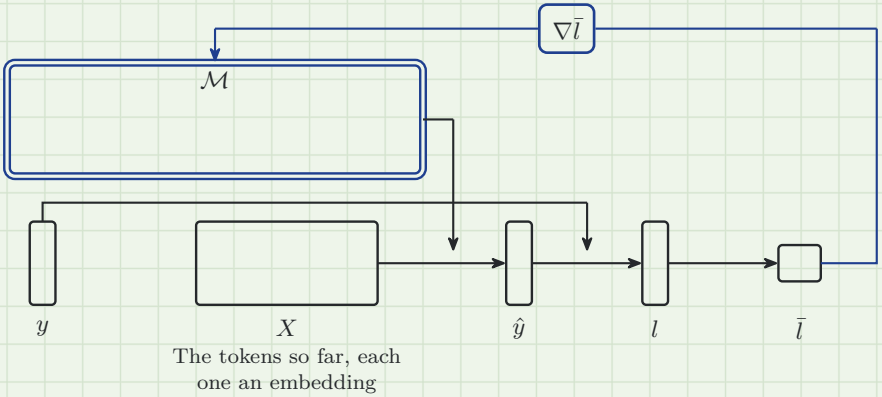
A row is text with its next token cut off. Every position of every document gives one.

The box holds a transformer. Vaswani and others, *Attention is all you need*, 2017.



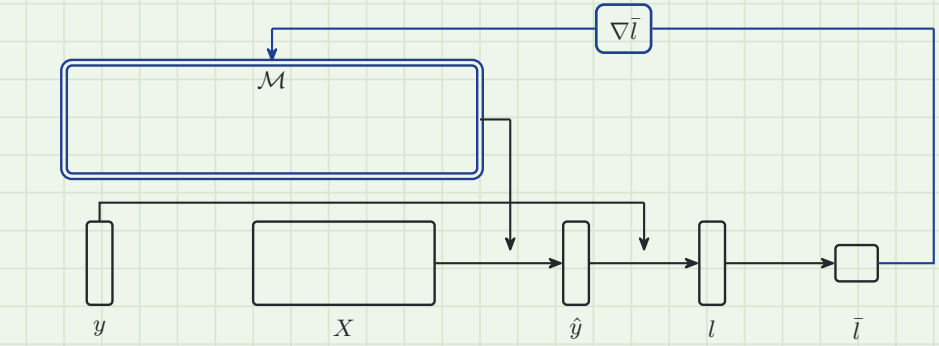
A row is text with its next token cut off. Every position of every document gives one.

The box holds a transformer. Vaswani and others, *Attention is all you need*, 2017.



A row is text with its next token cut off. Every position of every document gives one.

The box holds a transformer. Vaswani and others, *Attention is all you need*, 2017.

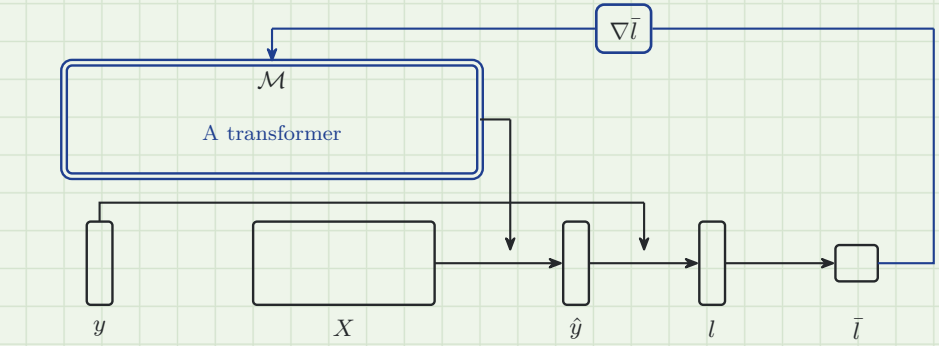


y
The token that actually came next

X
The tokens so far, each one an embedding

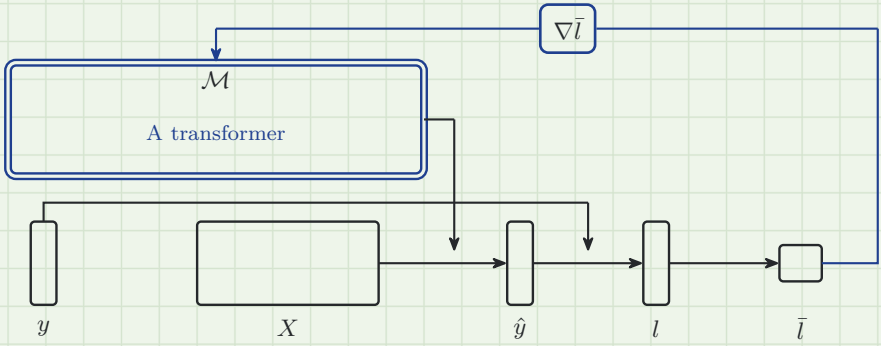
A row is text with its next token cut off. Every position of every document gives one.

The box holds a transformer. Vaswani and others, *Attention is all you need*, 2017.



A row is text with its next token cut off. Every position of every document gives one.

The box holds a transformer. Vaswani and others, *Attention is all you need*, 2017.



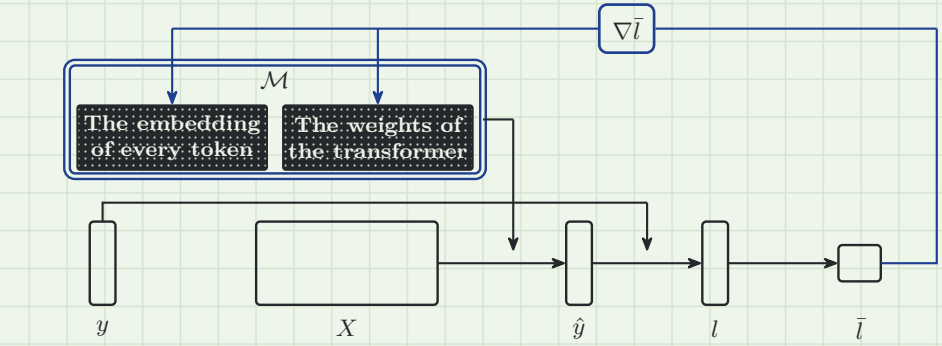
y
The token that actually came next

X
The tokens so far, each one an embedding

$\hat{y}$
The token the model expects

A row is text with its next token cut off. Every position of every document gives one.

The box holds a transformer. Vaswani and others, *Attention is all you need*, 2017.



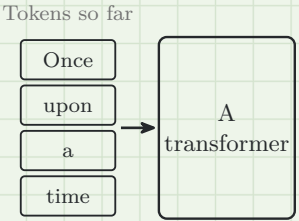
Tokens so far

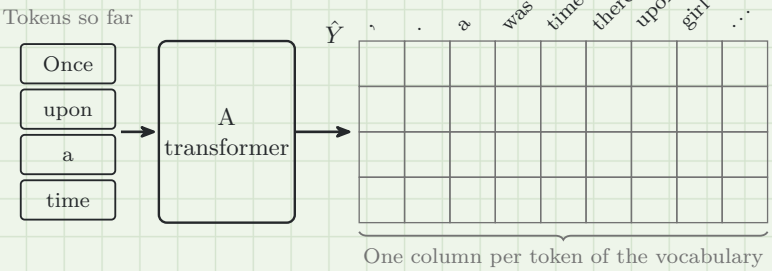
Once

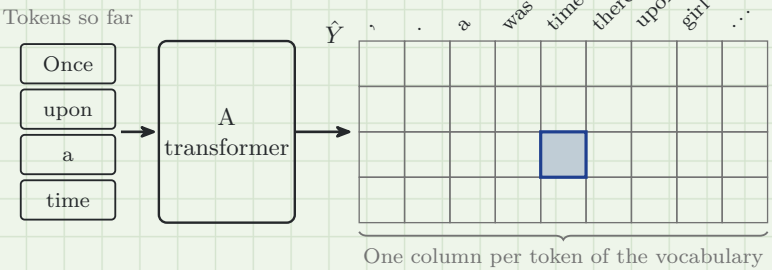
upon

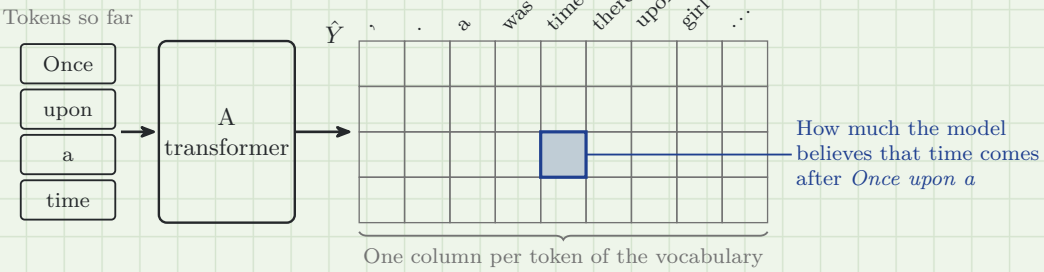
a

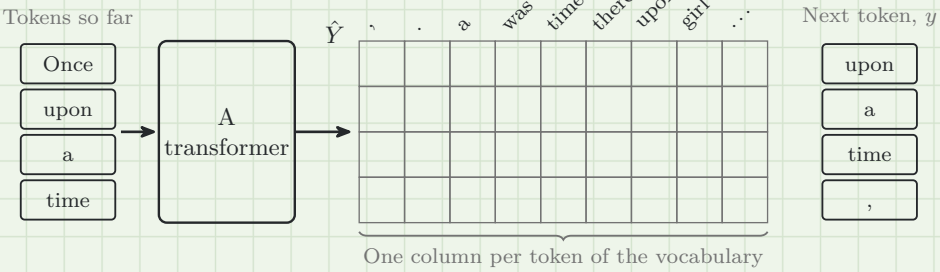
time

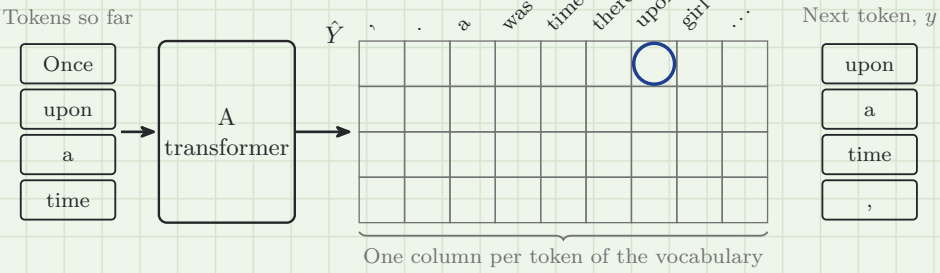

Tokens so far
Once
upon
a
time
A
transformer

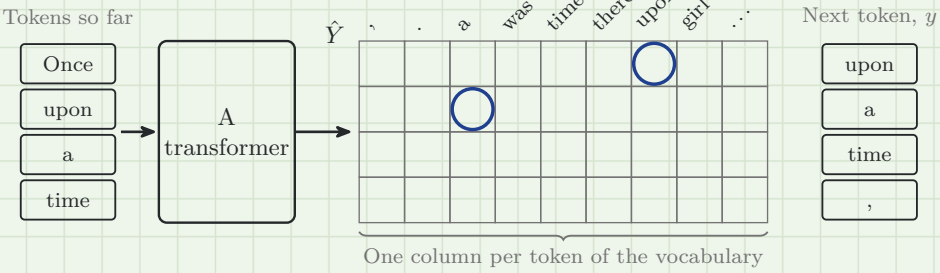

Tokens so far
Once
upon
a
time
A transformer
$\hat{Y}$
,
.
a
was
time
there
upon
girl
...
One column per token of the vocabulary

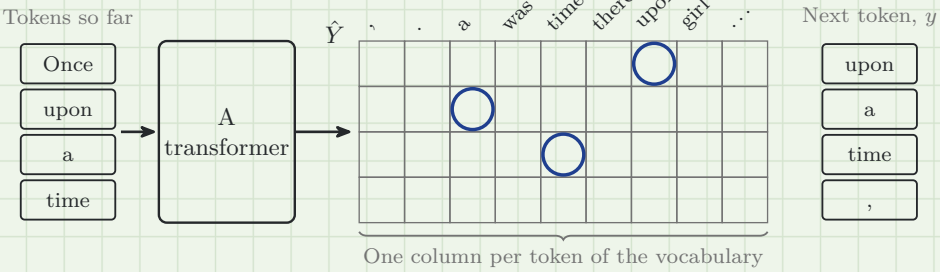

Tokens so far
Once
upon
a
time
A transformer
$\hat{Y}$
,
.
a
was
time
there
upon
girl
...
One column per token of the vocabulary

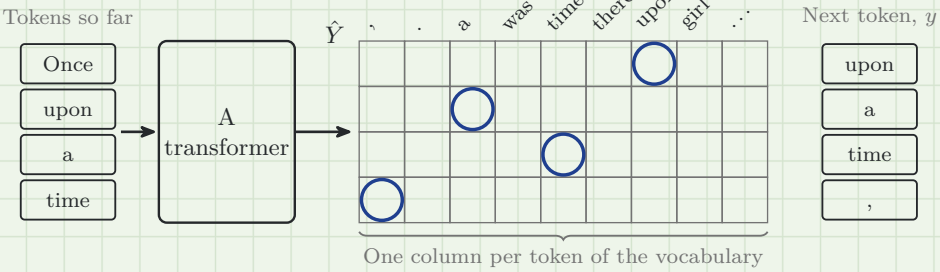

Tokens so far
Once
upon
a
time
A transformer
$\hat{Y}$
,
.
a
was
time
there
upon
girl
...
How much the model believes that time comes after *Once upon a*
One column per token of the vocabulary

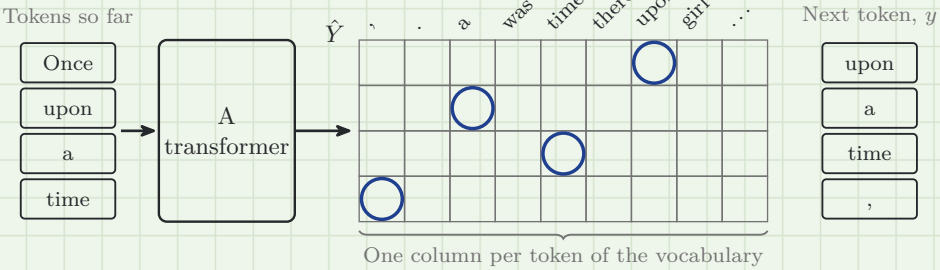

Tokens so far
Once
upon
a
time
A transformer
$\hat{Y}$
,
.
a
was
time
there
upon
girl
...
One column per token of the vocabulary
Next token, y
upon
a
time
,


Tokens so far
Once
upon
a
time
A transformer
$\hat{Y}$
,
.
a
was
time
there
upon
girl
...
One column per token of the vocabulary
Next token, y
upon
a
time
,

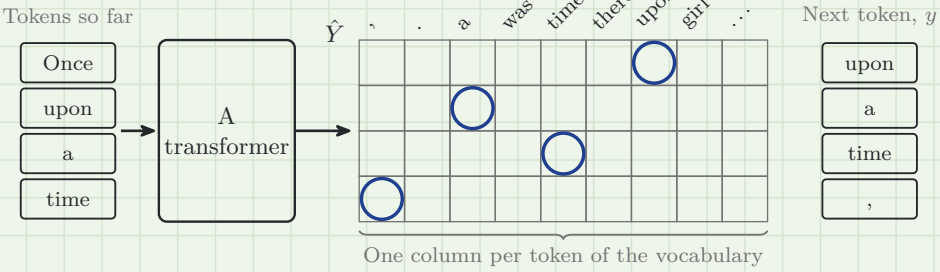

Tokens so far
Once
upon
a
time
A transformer
$\hat{Y}$
,
.
a
was
time
there
upon
girl
...
Next token, y
upon
a
time
,
One column per token of the vocabulary


Tokens so far
Once
upon
a
time
A transformer
$\hat{Y}$
,
.
a
was
time
there
upon
girl
...
Next token, y
upon
a
time
,
One column per token of the vocabulary

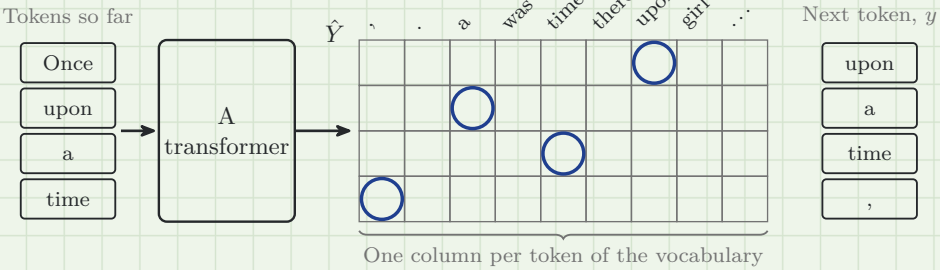

Tokens so far
Once
upon
a
time
A transformer
$\hat{Y}$
,
.
a
was
time
there
upon
girl
...
Next token, y
upon
a
time
,
One column per token of the vocabulary



$$\bar{l} = -\frac{1}{n}\left(\log \hat{y}_{1,\text{upon}}\right)$$



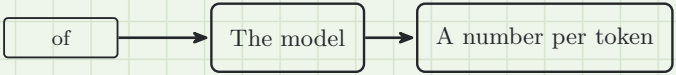
$$\bar{l} = -\tfrac{1}{n}\left(\log \hat{y}_{1,\text{upon}} + \log \hat{y}_{2,\text{a}}\right)$$



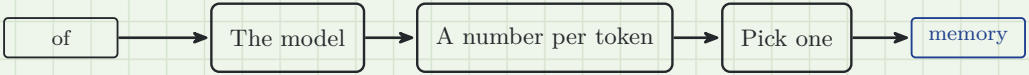
$$\bar{l} = -\tfrac{1}{n}\left(\log \hat{y}_{1,\text{upon}} + \log \hat{y}_{2,\text{a}} + \log \hat{y}_{3,\text{time}} + \cdots\right)$$

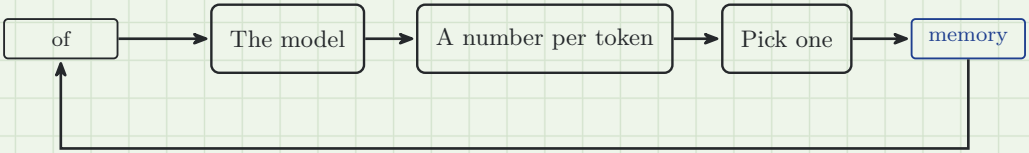
of


of
The model


of
The model
A number per token


of
The model
A number per token
Pick one


of
The model
A number per token
Pick one
memory


of
The model
A number per token
Pick one
memory
Appended to the prompt, and the whole thing runs again

Entry 8

Positional encodings

- Consider the sentences *the dog chased the cat* and *the cat chased the dog*.

- Consider the sentences *the dog chased the cat* and *the cat chased the dog*.
- They are permutations of each other, so the attention maps will also just be permutations of each other.

- Consider the sentences *the dog chased the cat* and *the cat chased the dog*.
- They are permutations of each other, so the attention maps will also just be permutations of each other.
- Note that attention mechanisms do not pay attention to the position of a token in the sentence.

[CLS]	0	0	0	0	0	0
because	1	0	0	0	0	0
it	0	1	0	0	0	0
was	1	1	0	0	0	0
two	0	0	1	0	0	0
tired	1	0	1	0	0	0
[EOS]	0	1	1	0	0	0

BP

[CLS]	0	0	0	0	0	0
because	1	0	0	0	0	0
it	0	1	0	0	0	0
was	1	1	0	0	0	0
two	0	0	1	0	0	0
tired	1	0	1	0	0	0
[EOS]	0	1	1	0	0	0

BP

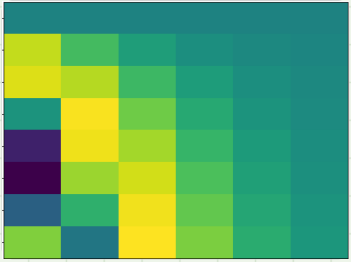
$$BP_{ij} = \left(i \div 2^j\right) \bmod 2$$
$$= \lfloor 2^{-j} i \rfloor \bmod 2$$

- We would like to create an embedding that encodes the position of the word in the sentence.
- A natural idea is to use a binary encoding of the position.
- Note that the entry i, j of this positional matrix has the form $f(\alpha^{-j} i)$, where $\alpha > 0$ is a scaling factor and f is a periodic function.
- In the binary positional encoding case, $f(x) = \lfloor x \rfloor \bmod 2$.

[CLS]	0	0	0	0	0	0
because	1	0	0	0	0	0
it	0	1	0	0	0	0
was	1	1	0	0	0	0
two	0	0	1	0	0	0
tired	1	0	1	0	0	0
[EOS]	0	1	1	0	0	0

BP

$$BP_{ij} = \left(i \div 2^j\right) \bmod 2$$
$$= \lfloor 2^{-j} i \rfloor \bmod 2$$

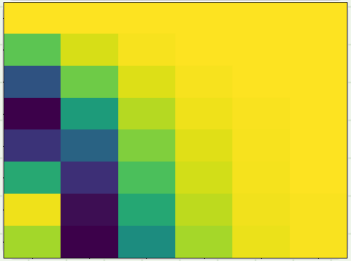


- We generalize this by using sin or cos instead of $f(x) = \lfloor x \rfloor \bmod 2$.

[CLS]	0	0	0	0	0	0
because	1	0	0	0	0	0
it	0	1	0	0	0	0
was	1	1	0	0	0	0
two	0	0	1	0	0	0
tired	1	0	1	0	0	0
[EOS]	0	1	1	0	0	0

BP

$$BP_{ij} = \left(i \div 2^j\right) \bmod 2$$
$$= \lfloor 2^{-j} i \rfloor \bmod 2$$

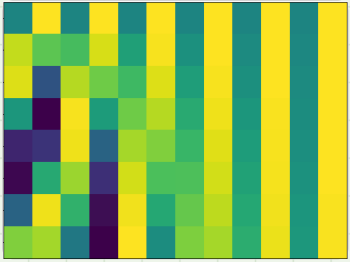


- We generalize this by using sin or cos instead of $f(x) = \lfloor x \rfloor \bmod 2$.

[CLS]	0	0	0	0	0	0
because	1	0	0	0	0	0
it	0	1	0	0	0	0
was	1	1	0	0	0	0
two	0	0	1	0	0	0
tired	1	0	1	0	0	0
[EOS]	0	1	1	0	0	0

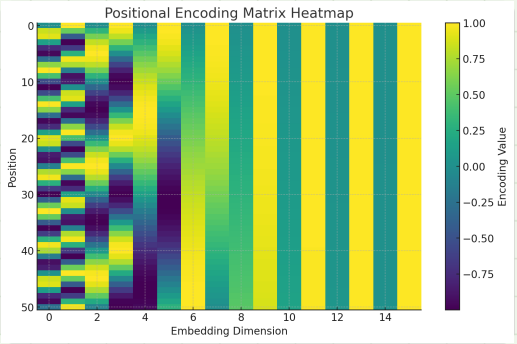
BP

$$BP_{ij} = \left(i \div 2^j\right) \bmod 2$$
$$= \lfloor 2^{-j} i \rfloor \bmod 2$$

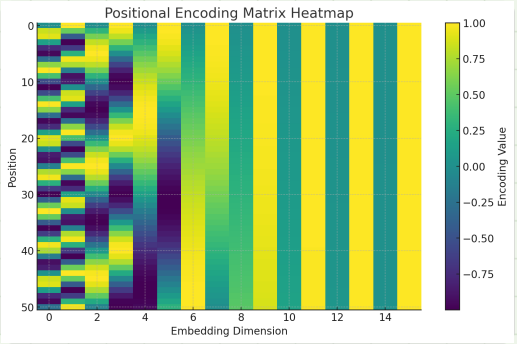


- We actually use both sin and cos and then interleave them.

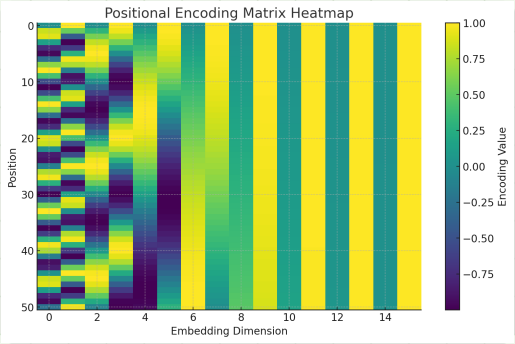
- The positional encoding matrix p has T rows and d columns.



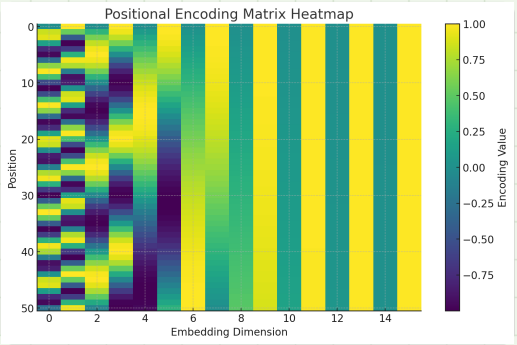
- The positional encoding matrix p has T rows and d columns.
- The value T is the maximum number of tokens in a sentence.



- The positional encoding matrix p has T rows and d columns.
- The value T is the maximum number of tokens in a sentence.
- The value d is the dimensionality of the positional embedding.



- The positional encoding matrix p has T rows and d columns.
- The value T is the maximum number of tokens in a sentence.
- The value d is the dimensionality of the positional embedding.
- The encoding of token i is defined through the formulas
  $p_{i,2j} = \sin\left(\alpha^{-2j} i\right)$ and
  $p_{i,2j+1} = \cos\left(\alpha^{-2j} i\right)$,
  for $j \leq d/2$ and $\alpha = 10^{4/d}$.



- Concatenation substantially increases the dimensionality of the embedding, increasing the number of parameters to be trained.

- Concatenation substantially increases the dimensionality of the embedding, increasing the number of parameters to be trained.
- With sufficient dimensionality, since the positional embeddings are somewhat statistically independent from the token embeddings, both embeddings are somewhat orthogonal. Hence, the addition still preserves information about the two vectors.

- Concatenation substantially increases the dimensionality of the embedding, increasing the number of parameters to be trained.
- With sufficient dimensionality, since the positional embeddings are somewhat statistically independent from the token embeddings, both embeddings are somewhat orthogonal. Hence, the addition still preserves information about the two vectors.
- Since most of the operations done in an attention mechanism are linear, the final embedding is somewhat the addition of the result of the operations applied to both the positional and the token embeddings.

Entry 9

Residual blocks

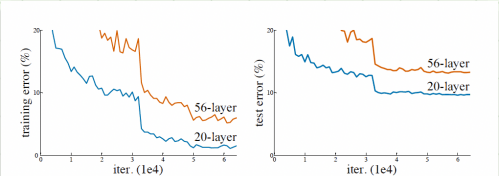


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

Figure 1 of Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun. Deep Residual Learning for Image Recognition. CVPR 2016.
https://arxiv.org/abs/1512.03385

- When deeper networks are able to start converging, a *degradation problem* appeared. Training performance degrades rapidly. This is not because of overfitting.

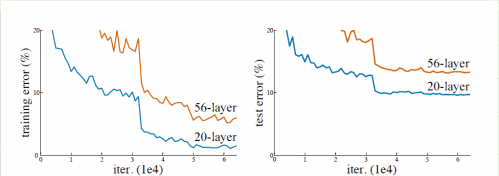


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

Figure 1 of Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun. Deep Residual Learning for Image Recognition. CVPR 2016.
`https://arxiv.org/abs/1512.03385`

- When deeper networks are able to start converging, a *degradation problem* appeared. Training performance degrades rapidly. This is not because of overfitting.
- The problem is mostly due to difficulties of information propagating, both forward and backward, through the layers in the network.

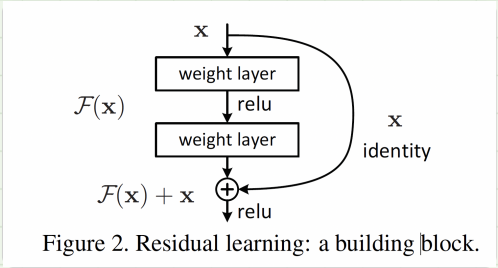


Figure 2. Residual learning: a building block.

Figure 2 of Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun. Deep Residual Learning for Image Recognition. CVPR 2016.
https://arxiv.org/abs/1512.03385

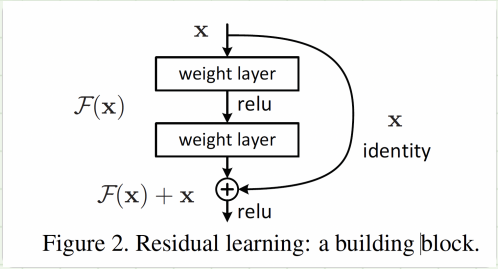


Figure 2. Residual learning: a building block.

Figure 2 of Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun. Deep Residual Learning for Image Recognition. CVPR 2016.
https://arxiv.org/abs/1512.03385

- To address this problem, residual blocks were introduced. They work by adding the output of a layer to the output of a few layers ahead.

Entry 10

Transformer architecture

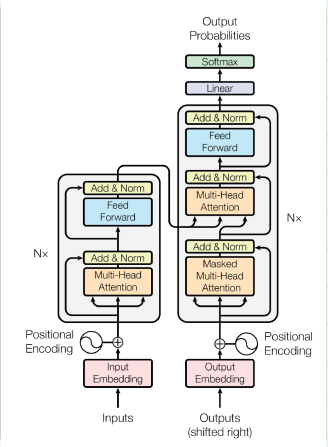


Figure 1 of

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser and Illia Polosukhin.

Attention Is All You Need.

Advances in Neural Information Processing Systems 30 (NeurIPS 2017).

`https://arxiv.org/abs/1706.03762`